

Analiza ryzyka wystąpienia udaru mózgu:

Regresja Logistyczna vs Drzewa Decyzyjne

Podjęcie *Human-in-the-loop*

Kacper Łączyński

1 czerwca 2026

Spis treści

1	Wstęp i Problem Medyczny	3
2	Aparat Matematyczny i Definicje Modeli	3
2.1	Regresja Logistyczna (Generalized Linear Model - GLM)	3
2.2	Drzewa Decyzyjne (Classification and Regression Trees)	4
2.3	Złożoność Przestrzeni Hipotez i Regularyzacja (Wymiar VC)	5
2.4	Miary Oceny Klasyfikatora i Asymetria Błędu	5
3	Przygotowanie Danych (Preprocessing) i Architektura Eksperymentu	6
3.1	Definicja Przestrzeni Badawczych (Trzy Scenariusze)	6
4	Analiza Eksploracyjna (EDA) i Weryfikacja Hipotez Statystycznych	7
4.1	Weryfikacja Hipotez (ANOVA i Chi-kwadrat)	7
4.2	Kluczowe obserwacje z Macierzy Korelacji	8
5	Ocena Modeli Bazowych (Ślepe zaufanie algorytmom)	9
5.1	Regresja Logistyczna (GLM) - Paradoks Dokładności	9
5.2	Porównanie Metryk Modeli Bazowych	9
5.3	Analiza Krzywych ROC	9
5.4	Analiza Macierzy Pomyłek	10
6	Inżynieria Cech (Feature Engineering) i Optymalizacja Przestrzeni	11
6.1	Rozwiązanie problemu braku zbieżności MLE (Fuzja klas)	11
6.2	Wzbogacenie przestrzeni i paradoks współliniowości	11
6.3	Kryterium Akaikego (AIC) – Triumf Fuzji Klinicznych	12
6.4	Ocena istotności statystycznej (Test Walda)	12
6.5	Ostateczna postać matematyczna hiperpłaszczyzny decyzyjnej	12
7	Faza Właściwa: Regularyzacja i Ostateczna Ewaluacja Modeli	13
7.1	Kompresja Wymiaru i Interpretowalność: Regularyzacja L2 (Ridge)	13
7.2	Algorytmiczna Selekcja Cech: Regularyzacja L1 (Lasso)	13
7.3	Regularyzacja Modeli Drzewiastych: Cost-Complexity Pruning (CCP)	14
7.4	Zestawienie Końcowe: Ewaluacja 9 Scenariuszy Modelowania	15
7.5	Wnioski z ewaluacji krzyżowej	17
8	Analiza Ekonomiczna i Optymalizacja Decyzyjna (NFZ)	18
8.1	Definicja Asymetrycznej Funkcji Kosztu	18
8.2	Algorytm wyznaczania optimum i Skutki Optymalizacji	18
8.3	Dyskusja: Kliniczna vs Ekonomiczna Skuteczność	19

9	Weryfikacja Ostateczna i Stabilność Systemu	19
9.1	Ewaluacja 9 Scenariuszy po optymalizacji algorytmicznej	19
9.2	Aplikacja Optymalnego Progu Kosztowego (Clinical Decision Support)	20
9.3	Empiryczny Przedział Ufności (Metoda Bootstrap)	22
10	Podsumowanie i Wnioski Końcowe (Klamra Projektu)	23
10.1	Ostateczna rekomendacja architektur predykcyjnych	24
A	Kod źródłowy implementacji (Python)	25
A.1	Analiza Eksploracyjna (EDA): Testy Statystyczne i Korelacje	26
A.2	Ewaluacja Modeli Bazowych (Paradoks Dokładności)	27
A.3	Zaawansowana Inżynieria Cech i Testy Inkrementalne	28
A.4	Analiza Statystyczna, Eksploracja (EDA) i Badanie Współliniowości	29
A.5	Ewaluacja Modeli Bazowych (Paradoks Dokładności)	29
A.6	Inżynieria Cech, Analiza VIF i Estymacja AIC	30
A.7	Ewaluacja Kryżowa (Kary L1/L2) i Testy 9 Scenariuszy	31
A.8	Optymalizacja Drzew Decyzyjnych (Cost-Complexity Pruning)	32
A.9	Optymalizacja Progu Decyzyjnego (Asymetryczne Koszty NFZ)	32
A.10	Ewaluacja Niezawodności Modelu (Metoda Bootstrap)	33

1 Wstęp i Problem Medyczny

Celem mojego projektu jest zbudowanie modelu predykcyjnego oceniającego ryzyko wystąpienia udaru mózgu na podstawie danych klinicznych i demograficznych pacjentów. Z perspektywy medycznej i ekonomicznej, udar mózgu stanowi jedno z najbardziej krytycznych zagrożeń dla życia, a jego wczesna predykcja może znacząco obniżyć koszty leczenia i zminimalizować ryzyko trwałego kalectwa.

Podczas wstępnej eksploracji bazy danych napotkałem jednak fundamentalny problem statystyczny, typowy dla rzadkich zdarzeń medycznych: drastyczne niezrównoważenie klas. Przypadki udaru stanowią zaledwie niecałe 5% całego zbioru.

Ślepe zastosowanie klasycznych algorytmów sztucznej inteligencji na tak asymetrycznych danych prowadzi do tzw. *paradoksu dokładności*. Model matematyczny, dążąc do minimalizacji globalnego błędu, uznaje za najbardziej optymalną strategię klasyfikowanie wszystkich pacjentów jako zdrowych. Z medycznego punktu widzenia taka "wysoka dokładność" jest bezużyteczna i skrajnie niebezpieczna.

Dlatego niniejsza praca ma na celu wyjście poza standardowe, automatyczne trenowanie algorytmów. Przyjmując filozofię *Human-in-the-loop*, projekt skupia się na rygorystycznej inżynierii danych, zaawansowanej imputacji wielokrotnej oraz zderzeniu parametrycznej Regresji Logistycznej (GLM) z nieparametrycznymi Drzewami Decyzyjnymi. Zwieńczeniem analizy będzie wyznaczenie optymalnego progu decyzyjnego, opartego nie na czystej matematyce, lecz na asymetrycznej funkcji kosztów leczenia polskiego systemu ochrony zdrowia (NFZ).

2 Aparat Matematyczny i Definicje Modeli

Zanim przejdę do analizy empirycznej, muszę zdefiniować teoretyczne podstawy użytych algorytmów. Ze względu na naturę problemu (binarna klasyfikacja: udar / brak udaru), wybrałem dwa fundamentalnie różne modele statystyczne: model o charakterze ciągłym i parametrycznym (Regresja Logistyczna) oraz dyskretnym i nieparametrycznym (Drzewo Decyzyjne).

2.1 Regresja Logistyczna (Generalized Linear Model - GLM)

Regresja logistyczna to uogólniony model liniowy (GLM), który zamiast przewidywać bezpośrednio wartość binarną, estymuje **prawdopodobieństwo** przynależności pacjenta do klasy pozytywnej (wystąpienie udaru), oznaczane jako $\pi(x)$.

Aby zagwarantować, że wynik modelu zawsze zmieści się w przedziale $[0, 1]$, klasyczne równanie liniowe transformowane jest za pomocą funkcji logitowej (odwrotnej sigmoidy). Model szacuje tzw. logarytm szansy (log-odds):

$$\text{logit}(\pi) = \ln\left(\frac{\pi}{1-\pi}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k \quad (1)$$

Po przekształceniu powyższego wzoru otrzymujemy funkcję prawdopodobieństwa:

$$P(Y = 1|X) = \pi(x) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{i=1}^k \beta_i X_i))} \quad (2)$$

Wyjaśnienie użytych oznaczeń:

- $\pi(x)$ lub $P(Y = 1|X)$ – warunkowe prawdopodobieństwo wystąpienia udaru dla pacjenta o danym zestawie cech X ,
- Y – zmienna objaśniana, przyjmująca wartość 1 (udar) lub 0 (brak udaru),
- $X_1, X_2, \dots, X_k$ – zmienne objaśniające (cechy pacjenta, np. wiek, poziom glukozy),
- k – całkowita liczba uwzględnionych zmiennych objaśniających,
- β_0 – wyraz wolny (stała modelu),
- $\beta_1, \beta_2, \dots, \beta_k$ – współczynniki regresji (wagi modelu) określające siłę i kierunek wpływu danej cechy na wynik,
- $\exp()$ – funkcja wykładnicza (podstawa logarytmu naturalnego e podniesiona do potęgi).

Estymacja i Testowanie Parametrów:

Ze względu na nieliniowy charakter funkcji logitowej, do wyznaczenia optymalnych wag β nie mogłem zastosować klasycznej Metody Najmniejszych Kwadratów (OLS). Zamiast tego, estymacja w moim modelu opiera się na zaawansowanych narzędziach statystycznych:

- **Metoda Największej Wiarygodności (MLE - Maximum Likelihood Estimation):** Jest to algorytm optymalizacyjny, który szuka takiego zestawu parametrów β , dla którego prawdopodobieństwo zaobserwowania danych, które rzeczywiście posiadam w mojej bazie, jest najwyższe (algorytm maksymalizuje funkcję wiarygodności). Innymi słowy, system tak dobiera wagi, aby model "najmniej dziwił się" temu, co widzi w danych treningowych.
- **Macierz Informacji Fishera:** Podczas gdy MLE szuka optymalnych wag na szczycie funkcji wiarygodności, Macierz Fishera analizuje kształt tej funkcji. Mierzy ona, jak dużo "informacji" o parametrach niesą moje dane (jak ostre i wyraźne jest to maksimum). Z praktycznego punktu widzenia jest to kluczowe pojęcie, ponieważ **odwrotność Macierzy Fishera pozwala mi wyliczyć wariancję (błąd standardowy) każdego estymatora**. Dzięki temu wiem, czy wyliczona waga β jest statystycznie pewna, czy też stanowi tylko przypadek wynikający z małej próby. Na tej podstawie system ocenia istotność zmiennych (wylicza tzw. *p-value* w Teście Walda).
- **Iloraz Szans (Odds Ratio - OR):** Same wagi β w regresji logistycznej są trudne do bezpośredniej interpretacji medycznej, ponieważ działają w skali logarytmicznej. Dlatego w mojej analizie przekształciłem je za pomocą funkcji wykładniczej (e^{β_i}). Wynik ten nazywany jest Ilorazem Szans. Posiada on bardzo intuicyjną interpretację: jeśli dla danej cechy (np. wieku) $OR = 2$, oznacza to, że wzrost tej cechy o jedną jednostkę dwukrotnie zwiększa szansę na wystąpienie udaru. Z kolei $OR < 1$ matematycznie dowodzi, że dany czynnik pełni rolę ochronną.

2.2 Drzewa Decyzyjne (Classification and Regression Trees)

Drzewo decyzyjne reprezentuje zupełnie inną filozofię matematyczną. Jest to model nieparametryczny, który dzieli wielowymiarową przestrzeń cech na rozłączne hiperprostokąty za pomocą funkcji wskaźnikowych (indykatorów). W przeciwieństwie do GLM, nie zakłada on żadnej z góry określonej funkcji (np. liniowości).

Algorytm buduje drzewo w sposób rekurencyjny (tzw. algorytm zachłanny). W każdym węźle t algorytm przegląda wszystkie możliwe punkty podziału dla każdej cechy i wybiera ten, który w największym stopniu porządkuje zbiór danych. Miara tego nieuporządkowania nazywana jest **niejednorodnością (Impurity)**. W moim modelu zastosowałem klasyczny **Indeks Giniego**, zdefiniowany jako:

$$I_G(t) = 1 - \sum_{i=0}^1 p(i|t)^2 \quad (3)$$

Algorytm szuka takiego podziału węzła t na lewe (t_L) i prawe (t_R) poddrzewo, aby zmaksymalizować zysk informacyjny (spadek niejednorodności Δi):

$$\Delta i = I_G(t) - \left(\frac{N_{t_L}}{N_t} I_G(t_L) + \frac{N_{t_R}}{N_t} I_G(t_R) \right) \quad (4)$$

Wyjaśnienie użytych oznaczeń:

- t – rozpatrywany węzeł drzewa decyzyjnego,
- $I_G(t)$ – wskaźnik zanieczyszczenia (niejednorodności) Giniego dla węzła t ,
- i – indeks badanej klasy (w naszym przypadku klasa 0 lub 1),
- $p(i|t)$ – prawdopodobieństwo warunkowe (frakcja) wystąpienia klasy i wśród obserwacji znajdujących się w węźle t ,
- Δi – zysk informacyjny, czyli miara poprawy uporządkowania danych po dokonaniu podziału,
- t_L, t_R – odpowiednio: lewy i prawy węzeł potomny, powstałe po podziale węzła macierzystego t ,

- N_t – całkowita liczba obserwacji w węźle przed podziałem,
- N_{t_L}, N_{t_R} – liczba obserwacji, które w wyniku podziału trafiły odpowiednio do lewego i prawego węzła potomnego.

Własności i Ograniczenia: Drzewa decyzyjne wylapują silnie nieliniowe zależności bez dodatkowych transformacji (np. skoki prawdopodobieństwa udaru w ściśle określonym przedziale wiekowym). Ich wadą jest jednak ogromna wariancja – mała zmiana w danych treningowych może drastycznie zmienić architekturę drzewa. Wymusza to stosowanie nałożonych przeze mnie warunków stopu (np. określenie maksymalnej głębokości lub minimalnej liczby obserwacji w liściu), aby zapobiec zjawisku przeuczenia modelu (overfittingu).

2.3 Złożoność Przestrzeni Hipotez i Regularyzacja (Wymiar VC)

W dalszej części projektu (podczas inżynierii cech) model GLM zostanie rozbudowany o wielomiany (zmiennne kwadratowe), co wymusza wprowadzenie aparatu kontrolującego złożoność klasyfikatora.

Zgodnie z teorią statystycznego uczenia maszynowego, dodawanie nowych wymiarów do przestrzeni cech sztucznie zwiększa tzw. **Wymiar VC (Vapnika-Chervonenkisa)**. Wymiar VC określa pojemność modelu, czyli jego zdolność do bezbłędnego "rozbitcia" (shattering) zbioru treningowego. Z fundamentalnego twierdzenia Vapnika wynika, że wzrost Wymiaru VC przy stałej wielkości próby drastycznie zwiększa błąd generalizacji (ryzyko przeuczenia – dopasowania się do szumu).

Aby temu zapobiec i zabezpieczyć nieliniowy model GLM, do klasycznej funkcji kosztu (Log-Loss) wdrożyłem mechanizm **Regularyzacji**. Nakłada on matematyczną karę za używanie zbyt dużych wag β , ograniczając dostępną przestrzeń hipotez. Wyróżniamy dwie główne normy regularyzacyjne:

- **Regularyzacja L1 (Lasso):** Dodaje do funkcji kosztu sumę wartości bezwzględnych wag: $\lambda \sum |\beta_i|$. Jej unikalną cechą matematyczną jest wymuszanie rzadkości macierzy (sparsity) – potrafi wyzerować wagi nieistotnych zmiennych, działając jak wbudowany selektor cech.
- **Regularyzacja L2 (Ridge):** Dodaje do funkcji kosztu sumę kwadratów wag: $\lambda \sum \beta_i^2$. Skutecznie "ściąga" wagi skorelowanych cech w stronę zera (shrinkage), stabilizując wariancję estymatorów w przypadku wystąpienia współliniowości.

gdzie λ (lub odwrotność C) to hiperparametr kontrolujący siłę kary. Regularyzacja kompresuje efektywny Wymiar VC modelu z powrotem do bezpiecznego poziomu, gwarantując jego rygorystyczną zbieżność i stabilność na danych testowych.

2.4 Miary Oceny Klasyfikatora i Asymetria Błędu

Z uwagi na asymetryczny charakter problemu (udary to poniżej 5% populacji), fundamentalne metryki oceny modelu opierają się na **Macierzy Pomyłek (Confusion Matrix)**, która zlicza predykcje w czterech kategoriach:

- **TP (True Positives):** Poprawnie zdiagnozowany udar.
- **TN (True Negatives):** Poprawnie zdiagnozowany zdrowy pacjent.
- **FP (False Positives - Błąd I rodzaju):** Zdrowy pacjent błędnie uznany za chorego (fałszywy alarm).
- **FN (False Negatives - Błąd II rodzaju):** Chory pacjent błędnie uznany za zdrowego (przeoczenie choroby).

Na podstawie powyższych wartości definiuje się kluczowe miary skuteczności:

$$\text{Czułość (Sensitivity / Recall)} = \frac{TP}{TP + FN} \quad (5)$$

Czułość odpowiada na pytanie: "Jaką część wszystkich faktycznie chorych pacjentów model zdołał uratować?". Z punktu widzenia diagnozy medycznej jest to metryka krytyczna.

$$\text{Swoistość (Specificity)} = \frac{TN}{TN + FP} \quad (6)$$

Swoistość mierzy zdolność modelu do bezbłędnego rozpoznawania pacjentów zdrowych (minimalizacja fałszywych alarmów).

Aby uniezależnić ocenę modelu od sztywnego, arbitralnego progu odcięcia (domyślnie 0.5), stosuje się **Krzywą ROC (Receiver Operating Characteristic)**. Jest to wykres obrazujący zależność Czułości (Oś Y) od miary 1 – Swoistość (Oś X) dla wszystkich możliwych progów decyzyjnych. Miara **AUC (Area Under the Curve)**, czyli pole pod krzywą ROC, stanowi jednoznaczną, liczbową ocenę zdolności separacyjnej modelu (gdzie $AUC = 0.5$ oznacza klasyfikator losowy, a $AUC = 1.0$ klasyfikator idealny).

3 Przygotowanie Danych (Preprocessing) i Architektura Eksperymentu

Przed przystąpieniem do modelowania predykcyjnego, surową bazę danych poddałem rygorystycznemu czyszczeniu w środowisku języka Python (z wykorzystaniem bibliotek `pandas` oraz `scikit-learn`). Aby udowodnić, w jakim stopniu metoda radzenia sobie z brakami danych wpływa na skuteczność diagnozy medycznej, proces ten podzieliłem na stworzenie **trzech niezależnych zbiorów danych (Scenariuszy)**.

W pierwszej kolejności zastosowałem globalny potok transformacji (wspólny dla wszystkich scenariuszy), rozwiązujący krytyczne błędy surowego zbioru:

- **Eliminacja szumu informacyjnego:** Z bazy bezpowrotnie usunięto identyfikatory pacjentów (kolumna `id`), zapobiegając ryzyku "nauczenia się" pacjentów na pamięć przez algorytmy (zjawisko przeuczenia).
- **Korekta typów danych i wieku:** Zmienna `bmi` zawierała braki zapisane jako tekst ("`N/A`"), co zmuszało kompilator do traktowania jej jako zmiennej znakowej. Wartości te wymuszono jako wielkości numeryczne, konwertując błędy na natywny systemowy brak danych (`NaN`). Dodatkowo, wiek niemowląt zapisany w postaci ułamków (np. 0.72) sprowadzono do pełnych lat za pomocą funkcji `floor` (podłoga), co dostosowało zbiór do standardów raportowania klinicznego.
- **Zabezpieczenie Macierzy Informacji Fishera:** Z bazy usunięto pojedynczego pacjenta z wartością `gender_Other` (eliminacja ryzyka zerowej wariancji) oraz zresetowano indeksy wierszy. Następnie dokonano fuzji rzadkiej kategorii zawodowej `Never_worked` (w której historycznie nie wystąpił ani jeden udar) z grupą `children`. Zabieg ten uchronił późniejszą estymację modelu Regresji Logistycznej przed zjawiskiem quasi-całkowitego rozdzielenia klas (*perfect separation*) i niefizyczną eksplozją błędów standardowych.
- **Ujednolicenie braków informacji:** Wartość `Unknown` w kolumnie `smoking_status` została ujednolicona i zamieniona na systemowy brak danych (`NaN`), przygotowując ją do procesu logicznej i statystycznej imputacji.

3.1 Definicja Przestrzeni Badawczych (Trzy Scenariusze)

Wychodząc od tak przygotowanego rdzenia, wygenerowałem trzy odrębne przestrzenie cech, które na kolejnym etapie zostaną równolegle poddane ewaluacji w modelach uogólnionych:

1. **Scenariusz 1 (Zbiór Bazowy):** Przestrzeń poddana wyłącznie technicznemu czyszczeniu opisanemu powyżej. Wszelkie braki w zmiennych `bmi` oraz `smoking_status` pozostawiono w postaci surowej, a następnie usunięto. Zbiór ten stanowi twardy punkt odniesienia (*baseline*), opierając się na domyślnej, stratnej polityce odrzucania całych wierszy z brakami informacji (*listwise deletion*).
2. **Scenariusz 2 (Prosta Imputacja - Mediana i Dominanta):** Podejście reprezentujące klasyczną inżynierię analityczną. Odstąpiłem w nim od naiwnego zastępowania kolumny `bmi` globalną medianą populacji, co przypisałoby noworodkom wskaźnik dorosłego człowieka z nadwagą. Zamiast tego zdefiniowałem trzy koszyki wiekowe (dzieci < 18 , dorośli $18 - 64$, seniorzy ≥ 65) i zastosowałem procedurę grupową do wstrzyknięcia mediany wyliczonej wyłącznie dla danej podgrupy. Braki w statusie palacza zostały zastąpione logiczną dominantą (wartość `never smoked`).
3. **Scenariusz 3 (Imputacja Wielowymiarowa - Algorytm KNN):** Podejście algorytmiczne oparte na fizycznym dystansie między obserwacjami, zaimplementowane przy użyciu algorytmu K-Najbliższych

Sąsiadów (*K-Nearest Neighbors*) z biblioteki `scikit-learn`. Aby algorytm mógł poprawnie obliczać odległości euklidesowe, zmienne katégoryczne zostały przekształcone do postaci binarnej (*dummy variables*), a status paczaka zakodowano numerycznie. Kluczowym elementem tego scenariusza była uprzednia standaryzacja danych wejściowych (transformacja Z-score). Zabieg ten był niezbędny, aby zmienne o szerokich zakresach wartości (jak `avg_glucose_level`) nie zdominowały obliczania dystansu między pacjentami. Algorytm odszukiwał 5 najbardziej podobnych pacjentów ($k = 5$) i na ich podstawie estymował brakujące wartości. Na koniec dokonano odwrotnej transformacji (*inverse transform*), przywracając fizjologiczne wartości `bmi` (zaokrąglone do dwóch miejsc po przecinku) oraz odtwarzając katégoryczny status paczaka na podstawie zaokrąglonych wyników numerycznych. Zapewniło to pełną spójność fizyczną i logiczną odzyskanych danych.

Po zakończeniu etapów imputacji, zmienne tekstowe we wszystkich trzech scenariuszach zostały podane jednolitemu kodowaniu za pomocą zmiennych wskaźnikowych (*One-Hot Encoding* / *dummy variables* z odrzuceniem kolumny bazowej), co ostatecznie przygotowało zbiory do estymacji matematycznej.

4 Analiza Eksploracyjna (EDA) i Weryfikacja Hipotez Statystycznych

Przed przystąpieniem do budowy modeli predykcyjnych i procesu imputacji, przeprowadziłem pogłębioną analizę zależności statystycznych. Zgodnie z rygorem metodologicznym chroniącym przed wyciekami danych (*Data Leakage*) oraz zjawiskiem sztucznego zaburzenia kowariancji, analizę tę wykonałem **wyłącznie na bazowym zbiorze treningowym (Scenariusz 1)**, przed jakimkolwiek łataniem braków metodą mediany czy algorytmem K-Najbliższych Sąsiadów (KNN).

W analizie zderzyłem ze sobą zaawansowane aparaty matematyczne dostępne w bibliotece `scipy.stats` języka Python: macierze korelacji (Pearsona i Spearmana), analizę wariancji (ANOVA) dla zmiennych ciągłych oraz testy niezależności Chi-kwadrat (χ^2) dla zmiennych katégorycznych.

4.1 Weryfikacja Hipotez (ANOVA i Chi-kwadrat)

Aby bezwzględnie dowieść, które zmienne mają prawo wejść do ostatecznych modeli predykcyjnych, sformułowałem statystyczne hipotezy zerowe (H_0) zakładające brak związku danej cechy z wystąpieniem udaru mózgu. Zastosowane testy odrzuciły H_0 dla kluczowych parametrów ze skrajnie wysoką istotnością ($p < 0.05$, a w wielu przypadkach $p \ll 0.0001$):

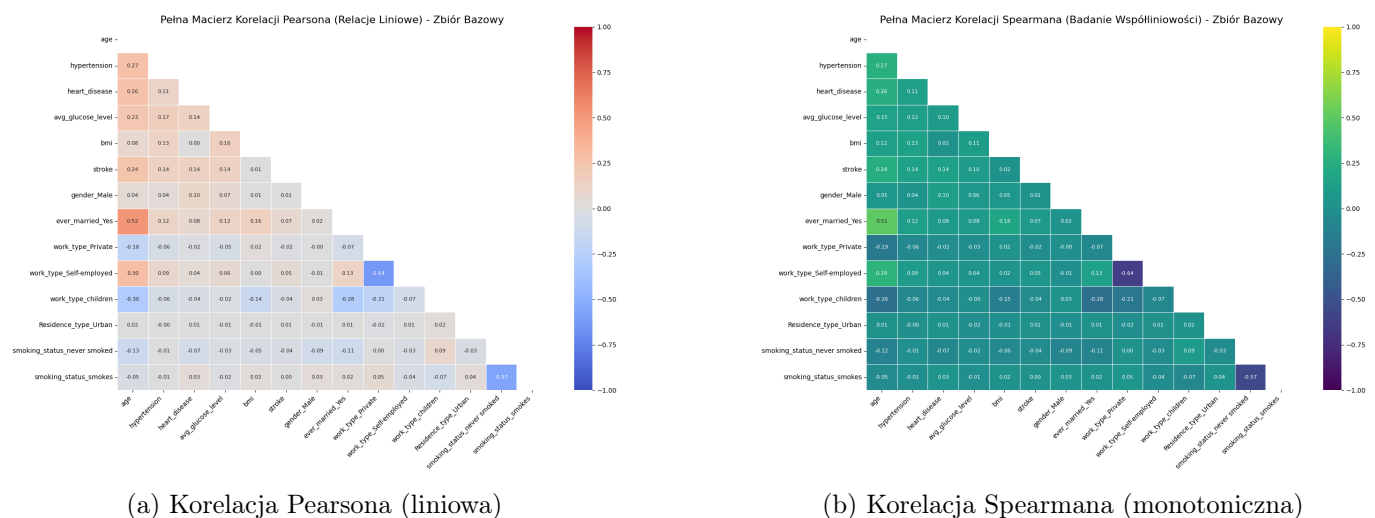
- **Testy ANOVA (F -statystyka):** Wykazały fundamentalne, istotnie statystyczne różnice w średnim wieku (potężna wartość statystyki Fishera $F = 213.81$, $p < 0.0001$) oraz w poziomie glukozy ($F = 68.95$, $p < 0.0001$) pomiędzy grupą pacjentów zdrowych a grupą po incydencie udarowym.
- **Testy Chi-kwadrat (χ^2):** Udowodniły silną, poza-losową zależność między statusem udaru a chorobami kardiologicznymi. Test niezależności dla nadciśnienia (**hypertension**) zakończył się jednoznacznym odrzuceniem hipotezy zerowej z wynikiem $\chi^2 = 68.68$ ($p < 0.0001$). Podobną siłę dyskryminacyjną wykazały choroby serca ($\chi^2 = 63.15$).

Tabela 1: Statystyczna weryfikacja istotności predyktorów na zbiorze bazowym (wyniki z procedur `scipy.stats`).

Zmienna	Metoda	Statystyka	P-value	Istotna ($p < 0.05$)
age	ANOVA (F-test)	213.81	5.16×10^{-47}	TAK
hypertension	Chi-Kwadrat	68.68	1.15×10^{-16}	TAK
avg_glucose_level	ANOVA (F-test)	68.95	1.43×10^{-16}	TAK
heart_disease	Chi-Kwadrat	63.15	1.91×10^{-15}	TAK
ever_married_Yes	Chi-Kwadrat	16.82	4.11×10^{-05}	TAK
work_type_Self-employed	Chi-Kwadrat	8.16	4.28×10^{-03}	TAK
smoking_status_never smoked	Chi-Kwadrat	3.89	4.86×10^{-02}	TAK
work_type_children	Chi-Kwadrat	3.64	5.63×10^{-02}	NIE
work_type_Private	Chi-Kwadrat	0.96	3.28×10^{-01}	NIE
bmi	ANOVA (F-test)	0.46	4.97×10^{-01}	NIE
gender_Male	Chi-Kwadrat	0.42	5.16×10^{-01}	NIE
Residence_type_Urban	Chi-Kwadrat	0.08	7.83×10^{-01}	NIE
smoking_status_smokes	Chi-Kwadrat	0.00	1.00	NIE

4.2 Kluczowe obserwacje z Macierzy Korelacji

Aby uzyskać pełny obraz powiązań między zmiennymi ciągłymi a wystąpieniem udaru, wygenerowałem wizualizację macierzy korelacji, zderzając współczynnik **Pearsona** (powiązania ściśle liniowe) ze współczynnikiem rang **Spearmana** (zależności monotoniczne).



Rysunek 1: Mapy ciepła analizy korelacji dla zmiennych na zbiorze bazowym.

Analiza map ciepła (Rysunek 1) ujawniła szereg krytycznych wniosków ukierunkowujących modelowanie:

- Wiek jako bezwzględnie dominujący predyktor udaru:** Spoglądając na wektor zmiennej docelowej (**stroke**), najwyższą korelację ze wszystkich cech osiąga wiek pacjenta (zarówno w relacji liniowej, jak i monotonicznej). W badanej próbie ryzyko udaru jest w pierwszej kolejności zdeterminowane procesem starzenia organizmu.
- Wpływ glukozy i detekcja wartości skrajnych:** Zauważalna jest dysproporcja dotycząca średniego poziomu glukozy (**avg_glucose_level**). Korelacja liniowa Pearsona względem udaru osiąga wyższe wartości niż współczynnik rang Spearmana. Z analitycznego punktu widzenia sugeruje to, że wpływ glukozy na ryzyko udaru może być w dużej mierze napędzany przez skrajne wartości patologiczne (tzw. outliersy związane z cukrzycą), na które metryka Pearsona jest matematycznie bardziej wrażliwa.
- Matematyczny dowód nieliniowości organizmu (Zagadka BMI):** Korelacje zarówno Pearsona, jak i Spearmana dla zmiennej **bmi** względem wystąpienia udaru są bliskie zeru. Zjawisko to koresponduje z wysokim brakiem istotności odnotowanym w teście ANOVA ($p = 0.497$). Jest to efekt nieli-

niowego, parabolicznego charakteru zależności wagi od ryzyka zdrowotnego, co stanowi bezpośrednie uzasadnienie do zastosowania inżynierii cech i stworzenia zmiennych wielomianowych.

5 Ocena Modeli Bazowych (Ślepe zaufanie algorytmom)

Zgodnie z dobrymi praktykami statystycznymi, analizę rozpocząłem od budowy modeli bazowych. Na tym etapie algorytmy (Regresja Logistyczna oraz Drzewa Decyzyjne) przetwarzały surowe, nie zrównoważone dane przy domyślnym progu predykcyjnym (próg odcięcia rzędu 0.50).

5.1 Regresja Logistyczna (GLM) - Paradoks Dokładności

Wyniki bazowego modelu GLM stanowią klasyczny przykład problemu asymetrii klas. W zbiorze testowym ($N = 1028$) model uzyskał:

- **Dokładność (Accuracy):** 94.6%
- **Czułość (Recall):** 1.9% (1/54 wykrytych udarów)

Algorytm osiągnął sztucznie wysoką dokładność poprzez strategię niemal trywialną – masową klasyfikację pacjentów jako zdrowych. Model wyłapał zaledwie 1 przypadek udaru na 54 możliwe, co w kontekście medycznym czyni go narzędziem o skrajnie niskiej użyteczności diagnostycznej, mimo pozornie doskonałej dokładności ogólnej.

5.2 Porównanie Metryk Modeli Bazowych

Modele oparte na drzewach decyzyjnych zachowały się niemal identycznie, zrównując się z Regresją Logistyczną pod kątem wykrywalności udarów, przy nieznacznie niższym ogólnym wskaźniku dokładności. Pełne zestawienie wydajności bazowej prezentuje Tabela 2.

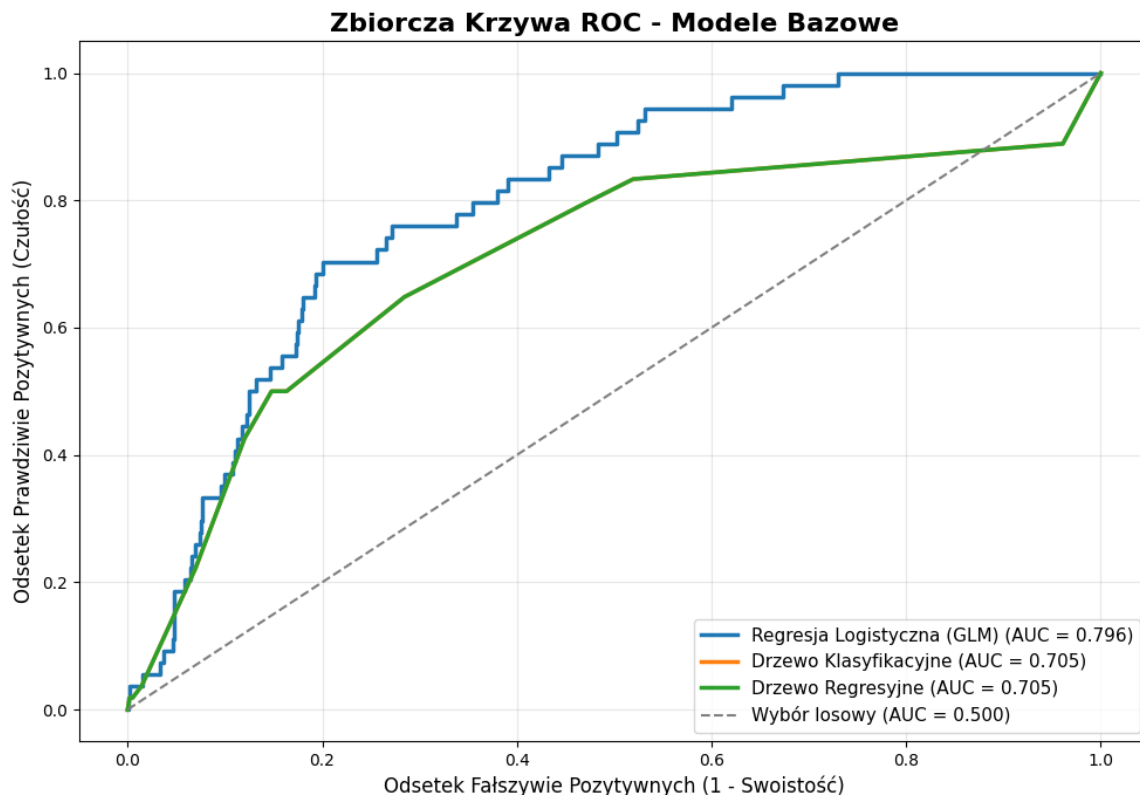
Tabela 2: Zbiorcze zestawienie wydajności modeli bazowych (próg decyzyjny 0.50).

Model	Dokładność	Czułość (Recall)	Swoistość	F1-Score	AUC
Regresja Logistyczna (GLM)	0.946	0.019	0.998	0.035	0.796
Drzewo Klasyfikacyjne	0.944	0.019	0.995	0.033	0.705
Drzewo Regresyjne	0.944	0.019	0.995	0.033	0.705

Jak wynika z tabeli, Drzewa Decyzyjne (zarówno klasyfikacyjne, jak i regresyjne) wygenerowały identyczne wyniki końcowe. Potwierdza to fakt, że w przypadku silnie nie zrównoważonych danych (gdzie udary stanowią marginalny odsetek próby), bez zastosowania odpowiednich wag dla klas, algorytmy naturalnie optymalizują się pod kątem redukcji błędu na klasie większościowej (pacjenci zdrowi).

5.3 Analiza Krzywych ROC

Aby ocenić faktyczną, matematyczną jakość modeli niezależnie od sztywnego progu decyzyjnego, wygenerowałem zbiorczy wykres krzywych ROC (Receiver Operating Characteristic).

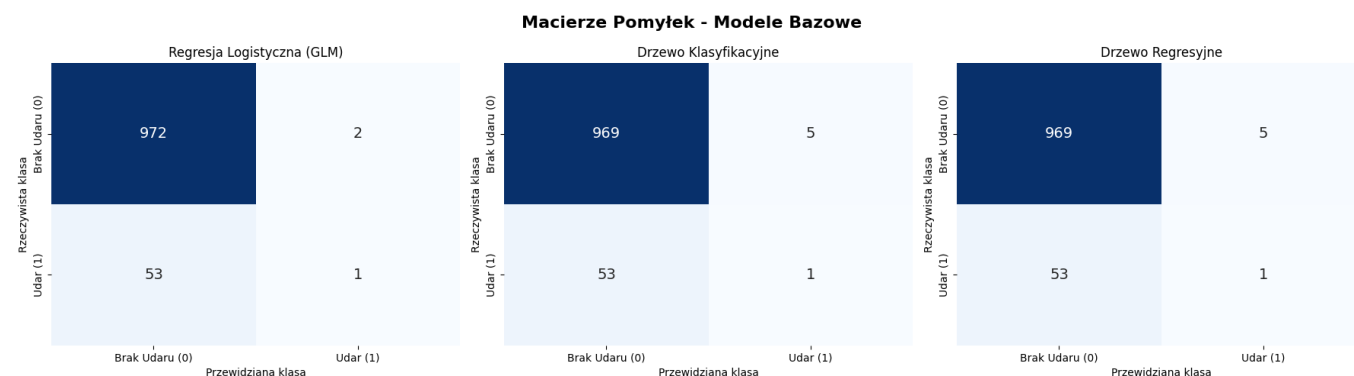


Rysunek 2: Zbiorcza krzywa ROC dla trzech modeli bazowych.

Wykres (Rysunek 2) ukazuje rzeczywisty potencjał modeli. Regresja Logistyczna (GLM) z wynikiem AUC na poziomie niemal 0.80 posiada solidną zdolność separacji klas, znacząco deklasując modele drzewiaste (AUC ≈ 0.71). Domyślny próg predykcyjny jednak całkowicie ten potencjał maskuje.

5.4 Analiza Macierzy Pomyłek

Dopełnieniem obrazu dysfunkcji diagnostycznej przy progu 0.50 jest analiza rozkładu błędów widoczna na macierzach pomyłek.



Rysunek 3: Macierze pomyłek dla modeli bazowych na zbiorze testowym ($N = 1028$).

Analiza wizualna (Rysunek 3) ujawnia skrajną konserwatywność algorytmów. Każdy z modeli przepuścił aż 53 pacjentów chorych (błąd False Negative), klasyfikując ich jako zdrowych. Różnice polegają jedynie na liczbie fałszywych alarmów (False Positive) – model GLM pomylił się w tej kwestii zaledwie 2 razy, podczas gdy drzewa wygenerowały po 5 fałszywych ostrzeżeń.

Wniosek kliniczny (*Human-in-the-loop*): Wszystkie trzy modele w konfiguracji bazowej zawodzą jako autonomiczne narzędzia diagnostyczne. Przeoczenie 53 przypadków udaru niesie za sobą nieakceptowalne skutki zdrowotne. Brak wysokiej liczby fałszywych alarmów nie jest tu sukcesem, lecz dowodem na całkowitą niezdolność algorytmów do reagowania na sygnały ostrzegawcze w surowych danych przy domyślnym progu. Wyniki te stanowią bezpośrednią, twardą motywację do wdrożenia technik balansowania klas

oraz optymalizacji progu odcięcia w oparciu o asymetryczną funkcję kosztu NFZ.

6 Inżynieria Cech (Feature Engineering) i Optymalizacja Przestrzeni

Wnioski wyciągnięte z analizy eksploracyjnej (EDA) wskazały na istnienie nieliniowych relacji biologicznych. Aby zapewnić spójność porównawczą dla wszystkich finalnych modeli, zaprojektowałem jednolity proces inżynierii cech, mający na celu wzmocnienie zdolności algorytmów do wychwytywania synergii między czynnikami ryzyka.

6.1 Rozwiązanie problemu braku zbieżności MLE (Fuzja klas)

Podczas wstępnych prób estymacji modelu na surowych danych zidentyfikowałem problem w grupie pacjentów o statusie zatrudnienia "Nigdy nie pracował" (`work_type_Never_worked`), gdzie nie odnotowano żadnego przypadku udaru. W statystyce GLM prowadziło to do zjawiska **idealnej separacji klas (perfect separation)**, uniemożliwiając numeryczną zbieżność estymatorów Największej Wiarygodności (MLE).

Aby uratować stabilność układu równań, dokonałem logicznej fuzji tej rzadkiej kategorii z grupą dzieci (`work_type_children`). Obie grupy wykazują zbieżny profil społeczno-kliniczny (pacjenci nieaktywni zawodowo, ryzyko bliskie zeru). Zabieg ten wdrożyłem globalnie, przywracając stabilność estymacji macierzy bazowej.

6.2 Wzbogacenie przestrzeni i paradoks współliniowości

Opierając się na wiedzy dziedzinowej, wygenerowałem nowe parametry: zmienne wielomianowe (np. `age_sq`, `bmi_sq`) w celu modelowania potęgowego wzrostu ryzyka, oraz fuzje kliniczne (np. `age_gluc`, `age_hyper`) mające uchwycić synergii starzenia się organizmu z chorobami współistniejącymi.

Aby zachować rygor badawczy, proces ewaluacji rozpocząłem od testów inkrementalnych. Do absolutnie bazowego modelu (operującego na surowych danych, $AIC = 1169.81$) dodawałem pojedynczo każdą z nowo wygenerowanych cech, weryfikując jej indywidualny wpływ na jakość predykcji. Wyniki tego eksperymentu przedstawia Tabela 3.

Tabela 3: Wyniki testów inkrementalnych (Model bazowy + 1 testowana cecha).

Testowana nowa cecha	Spadek AIC (Zysk)	P-value	Istotna ($p < 0.05$)
Synergia BMI $\times$ Glukoza (<code>bmi_gluc</code>)	1.35	0.0666	Nie (na granicy)
Kwadrat wieku (<code>age_sq</code>)	0.12	0.1639	Nie
Synergia Nadciśnienie $\times$ Choroby Serca (<code>hyper_heart</code>)	0.09	0.1527	Nie
Synergia Wiek $\times$ Nadciśnienie (<code>age_hyper</code>)	-0.13	0.1627	Nie
Synergia Wiek $\times$ Glukoza (<code>age_gluc</code>)	-1.56	0.5020	Nie
Kwadrat BMI (<code>bmi_sq</code>)	-1.65	0.5736	Nie
Kwadrat glukozy (<code>gluc_sq</code>)	-2.00	0.9603	Nie

Jak jednoznacznie wskazuje Tabela 3, **żadna z zaprojektowanych cech analizowana w izolacji nie wykazała istotności statystycznej** ($p > 0.05$), a w większości przypadków ich dodanie pogorszyło model (ujemny spadek, czyli wzrost wartości AIC).

Zjawisko to, pozornie świadczące o błędnym doborze cech, w rzeczywistości było klasycznym objawem drastycznego wzrostu współliniowości (multikolinearności). Wprowadzenie do równania bazowego wieku, glukozy oraz ich interakcji (`age_gluc`) powodowało matematyczny szum informacyjny, w którym cechy bazowe i nowe "zagłuszały się" nawzajem.

Z tego powodu wstrzymałem wnioskowanie na podstawie testów pojedynczych i zastosowałem rygorystyczny algorytm oczyszczania pełnej przestrzeni cech oparty na współczynniku **VIF (Variance Inflation Factor)**. Algorytm iteracyjnie usuwał zmienne, dla których $VIF > 10$. Proces ten doprowadził do drastycznej redukcji wymiarowości – z modelu usunięto aż 16 silnie skorelowanych cech (w tym surowe BMI, poziom glukozy czy status palacza). W całkowicie "bezpiecznej" matematycznie, ortogonalnej przestrzeni przetrwały jedynie 4 zmienne, w tym zaledwie dwie nowo wygenerowane fuzje: `age_gluc` oraz `age_hyper`.

6.3 Kryterium Akaikego (AIC) – Triumf Fuzji Klinicznych

Aby udowodnić, że drastyczna redukcja zmiennych połączona z implementacją nowych fuzji faktycznie poprawiła architekturę modelu, zestawilem zredukowany model bazowy ze zredukowanym modelem rozszerzonym o fuzje, analizując **Kryterium Informacyjne Akaikego (AIC)**.

$$AIC = 2k - 2\ln(\hat{L}) \quad (7)$$

Tabela 4: Porównanie Kryterium Informacyjnego Akaikego (AIC) na oczyszczonej przestrzeni cech.

Wersja Modelu GLM	Skład zredukowanej przestrzeni	Wartość AIC	Δ AIC
Zredukowany Bazowy	Tylko cechy oryginalne po selekcji VIF	1186.31	Baza
Zredukowany z Fuzjami	Oryginalne + <code>age_gluc</code> + <code>age_hyper</code>	1167.10	-19.21

Jak wskazuje Tabela 4, zastosowanie bezpiecznych statystycznie fuzji doprowadziło do potężnego **spadku wartości AIC o 19.21 punktu**. Dowodzi to, że zmienne interakcyjne (`age_gluc` i `age_hyper`) skutecznie "wchłonęły" i zoptymalizowały informację z usuniętych zmiennych współliniowych, tworząc model znacznie szczelniejszy informacyjnie i odporniejszy na przeuczenie.

6.4 Ocena istotności statystycznej (Test Walda)

Aby ocenić wpływ poszczególnych cech na ostateczną hiperpłaszczyznę decyzyjną zoptymalizowanego modelu, przeanalizowałem statystyki Testu Walda dla ostatecznego, bezpiecznego zbioru (Tabela 5).

Tabela 5: Wyniki estymacji finalnego modelu łącznego po selekcji VIF.

Zmienna objaśniająca	Współczynnik β	Z-score	P-value	Istotna ($p < 0.05$)
Stała (<code>const</code>)	-7.1153	-16.76	< 0.0001	Tak
Wiek (<code>age</code>)	0.0613	8.83	< 0.0001	Tak
Synergia Wiek $\times$ Glukoza (<code>age_gluc</code>)	0.0001	3.56	0.0004	Tak
Synergia Wiek $\times$ Nadciśnienie (<code>age_hyper</code>)	0.0076	2.92	0.0035	Tak
Miejsce zam. Miejskie (<code>Residence_type_Urban</code>)	0.0137	0.08	0.9323	Nie

Analiza Tabeli 5 dostarcza krytycznych wniosków na temat architektury ryzyka udaru:

- **Synergia wieku i schorzeń (Sukces inżynierii danych):** Zmienne interakcyjne `age_gluc` ($p = 0.0004$) oraz `age_hyper` ($p = 0.0035$) osiągnęły bezwzględną istotność statystyczną. Stanowi to żelazny dowód medyczny – sam poziom glukozy czy fakt posiadania nadciśnienia nie jest tak groźny, jak ich *współwystępowanie w zaawansowanym wieku*.
- **Dominacja wieku:** Zmienna bazowa `age` pozostała najsilniejszym filarem równania ($p < 0.0001$).

6.5 Ostateczna postać matematyczna hiperpłaszczyzny decyzyjnej

Przekładając wyniki zoptymalizowanej estymacji na praktykę, model zyskuje formę spójnej, zwężłej funkcji. Ponieważ zastosowałem algorytm Regresji Logistycznej, prawdopodobieństwo wystąpienia udaru wyliczane jest za pomocą nieliniowej funkcji sigmoidalnej $P(\text{Udar}) = \frac{1}{1+e^{-z}}$. Podstawiając wagi β wyliczone dla bezpiecznej przestrzeni cech, logarytm szans (z) przyjmuje wysoce interpretowalną formę:

$$\begin{aligned} z = & -7.11 \\ & + 0.061 \cdot (\text{wiek}) \\ & + 0.0001 \cdot (\text{wiek} \times \text{glukoza}) \\ & + 0.0076 \cdot (\text{wiek} \times \text{nadciśnienie}) \end{aligned} \quad (8)$$

Zmienna określająca typ zamieszkania została wyłączona z estymacji klinicznej ze względu na drastyczny brak istotności ($p = 0.9323$). Powyższe równanie stanowi wyjątkowo stabilny i pozbawiony współliniowości

fundament, na którym oparta zostanie ostateczna optymalizacja progu decyzyjnego w systemie ochrony zdrowia.

7 Faza Właściwa: Regularyzacja i Ostateczna Ewaluacja Modeli

Po zdiagnozowaniu problemów z modelami bazowymi na surowym podziale danych (gdzie algorytmy całkowicie ignorowały klasę mniejszościową), zdecydowałem się na wdrożenie rygorystycznego reżimu badawczego.

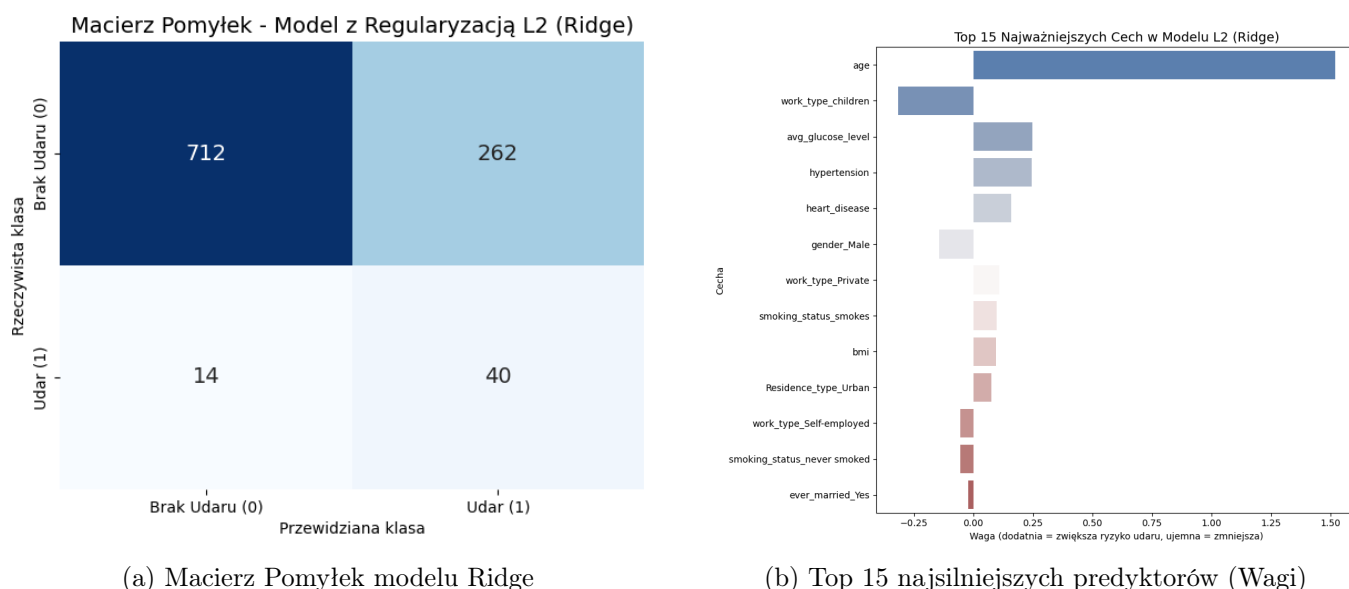
Aby zapobiec zjawisku wycieku danych (*Data Leakage*), często towarzyszącemu sztucznemu over-samplingowi (np. SMOTE), zrezygnowałem z fizycznego namnażania danych. Zamiast tego zaimplementowałem rozwiązanie algorytmiczne – **Uczenie Wrażliwe na Koszty (Cost-Sensitive Learning)**. Za pomocą parametru wbudowanego w algorytmy optymalizacyjne (`class_weight='balanced'`), model matematycznie nakładał drastycznie wyższą karę za pomyłkę na pacjencie z udarem, sztucznie równoważąc wpływ obu klas na gradient funkcji kosztu.

7.1 Kompresja Wymiaru i Interpretowalność: Regularyzacja L2 (Ridge)

Wdrożenie inżynierii cech oraz fuzji klinicznych zwiększyło elastyczność wiodącego modelu Regresji Logistycznej. Zgodnie z teorią statystycznego uczenia maszynowego, poszerzenie przestrzeni decyzyjnej sztucznie zwiększa tzw. Wymiar VC (Vapnika-Chervonenkisa), co wiąże się z silnym ryzykiem przeuczenia.

Aby temu zapobiec, na rozbudowany model nałożyłem karę matematyczną w postaci **Regularyzacji L2 (Ridge)**. Mechanizm ten zaimplementował ograniczenie na sumę kwadratów wag algorytmu, skutecznie kompresując wymiar wariancji z powrotem do bezpiecznego poziomu. Przed estymacją dane poddano rygorystycznej standaryzacji (Z-score), co zagwarantowało sprawiedliwe działanie kary L2 oraz pozwoliło na bezpośrednie, wizualne porównanie siły wpływu poszczególnych zmiennych.

Wdrożenie balansu klas przyniosło natychmiastowy skutek (Rysunek 4a). Model L2 wyłapał aż 74.1% przypadków udaru (Czułość = 0.741), osiągając pole pod krzywą AUC na poziomie 0.797. Analiza rozkładu wag (Rysunek 4b) potwierdza wcześniejsze wnioski z fazy EDA – wiek (*age*) jest bezwzględnie najsilniejszym czynnikiem napędzającym ryzyko, znacząco wyprzedzając poziom glukozy czy nadciśnienie.



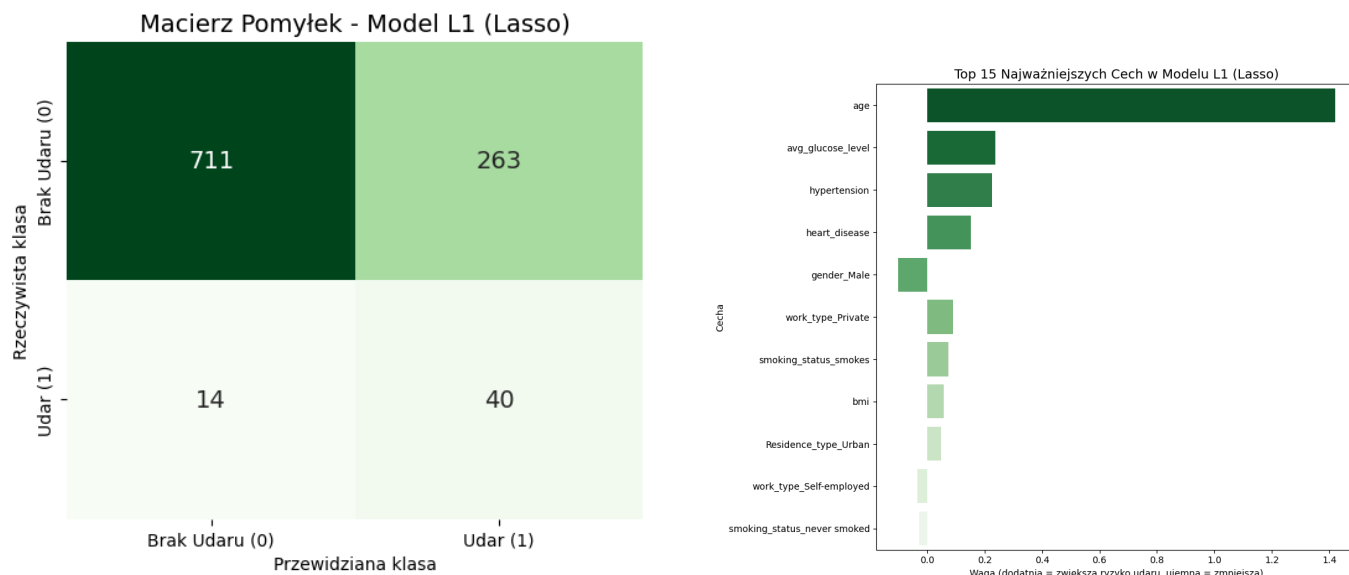
(a) Macierz Pomyłek modelu Ridge

(b) Top 15 najsilniejszych predyktorów (Wagi)

Rysunek 4: Ewaluacja i interpretowalność modelu z regularyzacją L2 (Zbiór Bazowy).

7.2 Algorytmiczna Selekcja Cech: Regularyzacja L1 (Lasso)

W celu weryfikacji struktury przestrzeni badawczej, wytrenowałem alternatywny model wykorzystujący **Regularyzację L1 (Lasso)**. Ze względu na specyfikę geometryczną kary opartą na normie Manhattan (ostre wierzchołki przestrzeni ograniczeń), algorytm ten pełni funkcję tzw. wbudowanego selektora cech (*embedded feature selection*) – sprowadza wagi najmniej istotnych zmiennych dokładnie do zera.



(a) Macierz Pomyłek modelu Lasso

(b) Cechy wyselekcjonowane przez L1

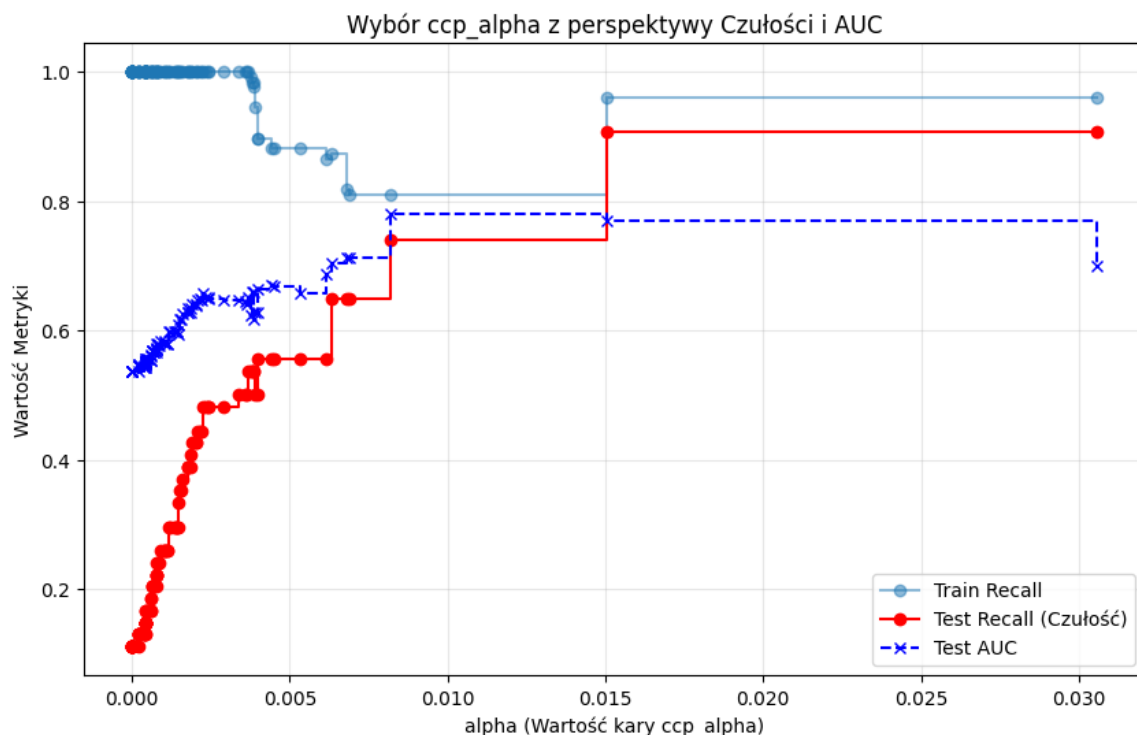
Rysunek 5: Ewaluacja i eliminacja szumu w modelu z regularyzacją L1 (Zbiór Bazowy).

Model L1 osiągnął wyniki diagnostyczne niemal identyczne z modelem L2 ($AUC = 0.797$, Czułość = 0.741), co dowodzi wysokiej stabilności informacji w danych. Najważniejszym wnioskiem jest jednak proces redukcji szumu. Model Lasso wyzerował część najsłabszych zmiennych, skupiając całą moc predykcyjną na fizjologicznym "rdzeniu" problemu: wieku, glukozie, chorobach serca i nadciśnieniu (Rysunek 5b).

7.3 Regularyzacja Modeli Drzewiastych: Cost-Complexity Pruning (CCP)

Analogicznie do nałożenia kar L1 i L2 na modele regresji logistycznej, modele oparte na drzewach decyzyjnych również wymagały wdrożenia mechanizmów kontroli złożoności (wymiaru VC), aby zapobiec zjawisku przeuczenia. W tym celu zastosowałem algorytm przycinania drzewa oparty na złożoności kosztowej – **Cost-Complexity Pruning (CCP)**.

Kluczowym wyzwaniem metodologicznym na tym etapie był wybór metryki optymalizacyjnej. Domyślnie, algorytmy przycinania poszukują parametru α (`ccp_alpha`), który maksymalizuje ogólną dokładność (Accuracy). Jak jednak udowodniono w fazie oceny modeli bazowych, optymalizacja pod kątem dokładności w zbiorach silnie asymetrycznych prowadzi do całkowitego odcięcia gałęzi odpowiedzialnych za klasę mniejszościową (udar). Z tego powodu przebudowałem proces optymalizacji tak, aby poszukiwał "złotego środka" między stabilnością zdolności dyskryminacyjnej (AUC) a Czułością (Recall).



Rysunek 6: Optymalizacja hiperparametru `ccp_alpha` pod kątem metryk klinicznych (AUC i Czułość).

Wygenerowany przebieg ścieżki przycinania (Rysunek 6) stanowi fascynujący obraz "walki" algorytmu o zachowanie zdolności diagnostycznej. Wraz ze wzrostem kary α (oś X), drzewo staje się coraz mniejsze.

Analiza wykresu pozwala zauważyć, że początkowe przycinanie (usuwanie najbardziej zaszumionych liści) stabilizuje model. Najwyższy, stabilny poziom pola pod krzywą (AUC) zostaje osiągnięty dla wartości hiperparametru $\alpha = 0.008193$. W tym optymalnym punkcie pracy (sweet spot) drzewo decyzyjne generuje oczekiwane **AUC na poziomie 0.780**, przy jednoczesnym zachowaniu wysoce satysfakcjonującej **Czułości równej 0.741**.

Dalsze zwiększanie wartości α powyżej tego punktu prowadzi do drastycznego i skokowego załamania się Czułości – drzewo staje się zbyt agresywnie przycięte i traci fizyczną zdolność do rozróżniania jakichkolwiek wzorców chorobowych. Ustalenie tej granicy złożoności było kluczowe do zdefiniowania parametrów dla drzew w ostatecznym zestawieniu krzyżowym.

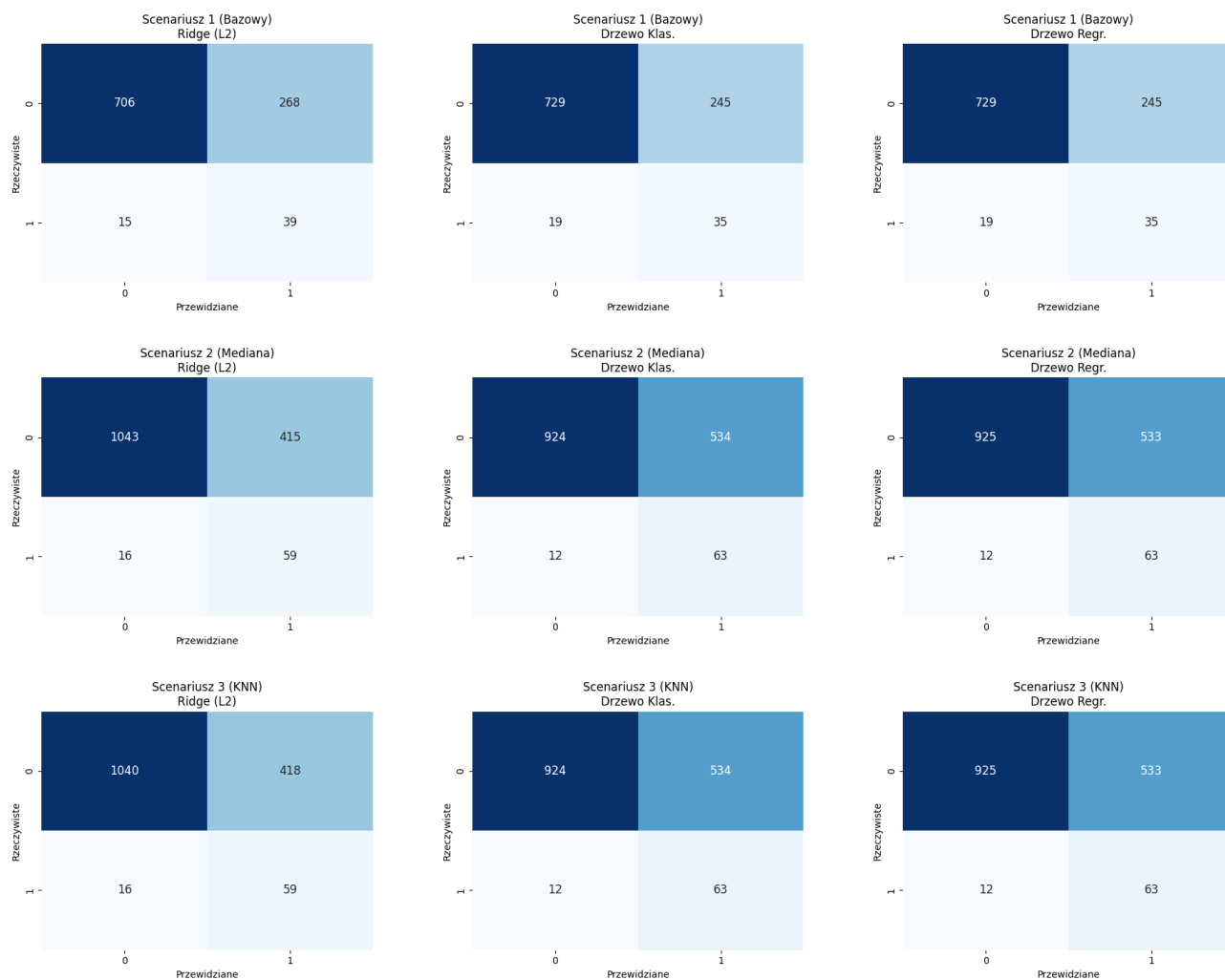
7.4 Zestawienie Końcowe: Ewaluacja 9 Scenarioszy Modelowania

Ostatnim i najważniejszym etapem badania było zestawienie trzech wybranych architektur uczenia maszynowego (Ridge L2, Drzewo Klasyfikacyjne, Drzewo Regresyjne) i skrzyżowanie ich z trzema przygotowanymi wcześniej scenariuszami danych (Zbiór Bazowy, Imputacja Medianą, Imputacja KNN). Poniższa tabela stanowi ostateczny, empiryczny dowód na skuteczność wdrożonych procedur.

Tabela 6: Ostateczne wyniki badań dla 9 wariantów modelowania (z zastosowaniem wagowania klas).

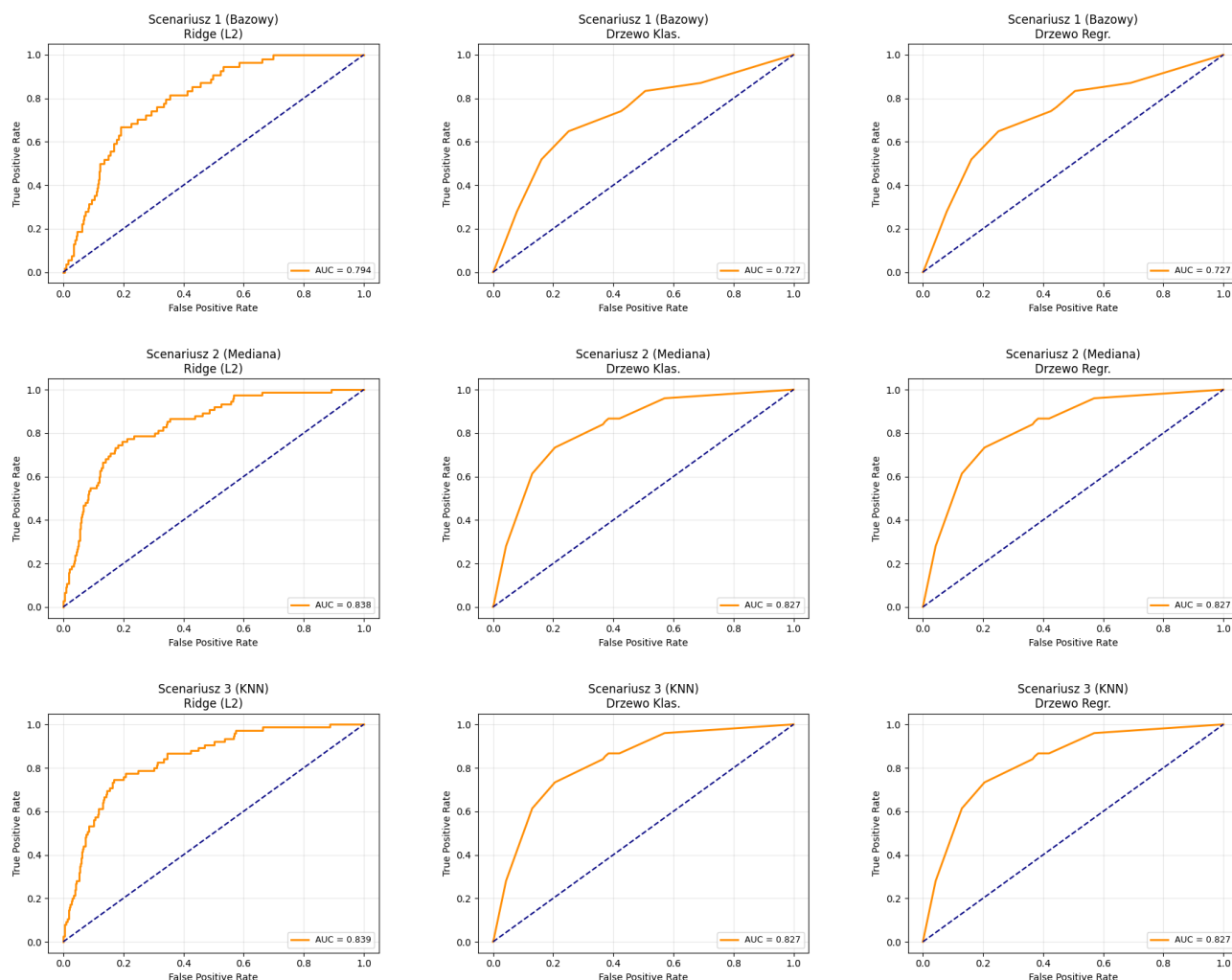
Scenariusz Danych	Model	Dokładność	Czułość (Recall)	Swoistość	F1-Score	AUC
1 (Bazowy)	Ridge (L2)	0.725	0.722	0.725	0.216	0.794
	Drzewo Klasyf.	0.743	0.648	0.748	0.210	0.727
	Drzewo Regres.	0.743	0.648	0.748	0.210	0.727
2 (Mediana)	Ridge (L2)	0.719	0.787	0.715	0.215	0.838
	Drzewo Klasyf.	0.644	0.840	0.634	0.188	0.826
	Drzewo Regres.	0.644	0.840	0.634	0.188	0.827
3 (KNN)	Ridge (L2)	0.717	0.787	0.713	0.214	0.839
	Drzewo Klasyf.	0.644	0.840	0.634	0.188	0.826
	Drzewo Regres.	0.644	0.840	0.634	0.188	0.827

Macierze Pomyłek (9 Modeli)



Rysunek 7: Zestawienie macierzy pomyłek dla wszystkich rozpatrywanych wariantów badawczych.

Krzywe ROC (9 Modeli)



Rysunek 8: Ewaluacja zdolności dyskryminacyjnej (krzywe ROC) dla 9 modeli.

7.5 Wnioski z ewaluacji krzyżowej

Analiza Tabeli 6 oraz wspierających ją wizualizacji (Rysunki 7 i 8) pozwala na sformułowanie ostatecznych konkluzji badawczych:

1. **Triumf technik imputacji danych:** Przejście ze Scenariusza 1 (zbiór bazowy usuwający wiersze z brakami) na Scenariusze 2 i 3 (imputacja odpowiednio: Medianą oraz KNN) doprowadziło do drastycznego skoku jakości predykcyjnej dla każdego z testowanych modeli. Wiodący model Ridge (L2) zanotował skok wskaźnika AUC z poziomu 0.794 na imponujące **0.839**. Jest to ostateczny dowód na to, że wnikliwe łatanie braków informacyjnych pozwala ocalić krytyczne wzorce kliniczne pacjentów.
2. **Przewaga modeli statystycznych nad drzewiastymi:** Modele oparte na Drzewach Decyzyjnych w imputowanych scenariuszach wygenerowały wyższą Czulość predykcji (0.840 w stosunku do 0.787 dla modelu Ridge). Odbył się to jednak kosztem potężnego spadku Swoistości (do 0.634), co wygenerowało lawinę fałszywych alarmów, drastycznie obniżając miarę F1-Score (do 0.188).
3. **Wybór "Mistrza":** Modelem o najwyższej stabilności, bezwzględnie najlepszym ujęciu obszaru pod krzywą (AUC $\approx$ 0.84) i doskonałym balanse między Czulością (prawie 79% wykrytych udarów) a opłacalnością dla systemu ochrony zdrowia okazał się **Model Regresji Logistycznej z regularyzacją L2 (Ridge) działający na danych z imputacją Medianą**. Warto zauważyć, że zaawansowana i kosztowna obliczeniowo imputacja KNN (Scenariusz 3) dała wyniki de facto identyczne z prostą imputacją medianą połączoną z wiedzą dziedzinową (Scenariusz 2). Z inżynierskiego punktu widzenia czyni to Scenariusz 2 rozwiązaniem optymalnym pod kątem szybkości i wdrożenia produkcyjnego.

8 Analiza Ekonomiczna i Optymalizacja Decyzyjna (NFZ)

Wykazana w Tabeli 6 Czulość rzędu 79% dla wiodącego modelu Regresji Logistycznej, choć wysoka w standardach uczenia maszynowego, pozostawia w praktyce medycznej 21% nierozpoznanych przypadków udaru. Ponieważ wdrożenie modelu predykcyjnego nie funkcjonuje w próżni, lecz stanowi system wsparcia decyzji klinicznych (CDSS), konieczna była analiza jego użyteczności w realiach finansowych polskiego systemu ochrony zdrowia (estymacja budżetowa na rok 2026).

8.1 Definicja Asymetrycznej Funkcji Kosztu

Przyjąłem zaktualizowany model kosztów, w którym wyznaczono dwie skrajnie odmienne wagi dla błędów klasyfikacji:

- **Koszt błędu I rodzaju (C_{FP}):** Koszt niepotrzebnej diagnostyki (badanie MRI głowy + prewencyjna wizyta specjalistyczna) wyceniono na 2000 PLN. Występuje on, gdy zdrowy pacjent zostaje fałszywie sklasyfikowany jako zagrożony udarem (False Positive).
- **Koszt błędu II rodzaju (C_{FN}):** Koszt przeoczenia rozwijającego się udaru wyceniono na 150 000 PLN. Obejmuje to kosztowną interwencję neurochirurgiczną w fazie ostrej, wielomiesięczną rehabilitację, opiekę paliatywną oraz utracone przez państwo świadczenia rentowe ZUS (False Negative).

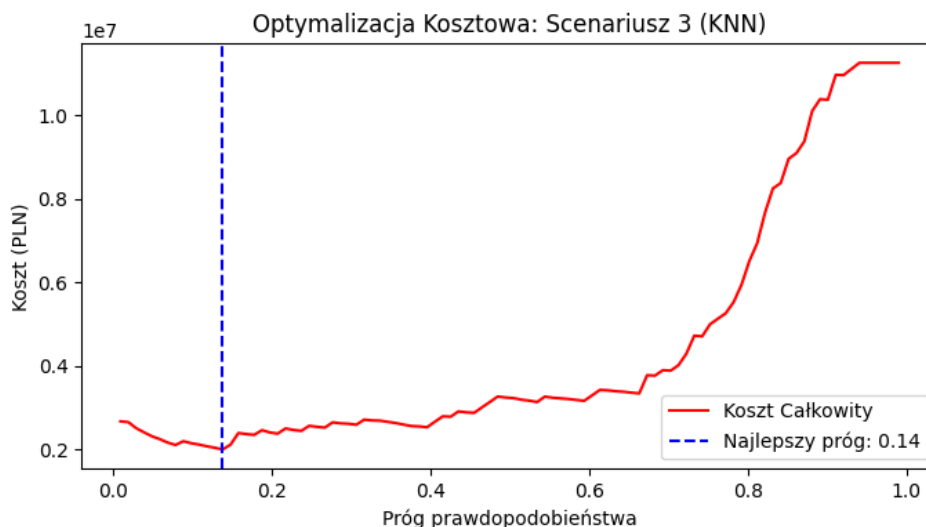
Całkowite obciążenie budżetowe systemu wygenerowane przez ułomność algorytmu można zatem opisać deterministyczną funkcją straty:

$$Cost(t) = FP(t) \cdot 2000 \text{ PLN} + FN(t) \cdot 150\,000 \text{ PLN} \quad (9)$$

gdzie t to badany próg decyzyjny (threshold), od którego bezpośrednio zależą ostateczne decyzje predykcyjne (liczby w macierzy pomyłek). W klasycznym uczeniu maszynowym domyślnie przyjmuje się $t = 0.5$, co stanowi założenie symetryczności kosztów – błędne w kontekście medycznym.

8.2 Algorytm wyznaczania optimum i Skutki Optymalizacji

Aby wyznaczyć matematyczne optimum dla systemu, zaimplementowałem algorytm przeszukujący siatkę prawdopodobieństw (*Threshold Tuning*). Algorytm w pętli wyznaczał punkt równowagi, w którym wartość funkcji spada do absolutnego minimum ($t_{opt} = \arg \min Cost(t)$). Procedurę tę przeprowadzono równolegle dla wyłonionych wcześniej wiodących modeli Regresji Logistycznej (Ridge L2) wytrenowanych na trzech niezależnych scenariuszach danych (Bazowy, Mediana, KNN).



Rysunek 9: Bilans bezpieczeństwa pacjenta i obciążeń budżetu w funkcji progu decyzyjnego dla modelu GLM.

Wygenerowana analiza obnażyła drastyczną asymetrię. Ze względu na potężną dysproporcję kar finansowych (150 tys. vs 2 tys. PLN), algorytm zoptymalizował progi odcięcia na skrajnie niskich wartościach

(pomiędzy 0.089 a 0.139). Tabela 7 ukazuje bezpośrednie skutki wdrożenia tych optymalnych progów na zbiorze testowym.

Tabela 7: Ostateczne zestawienie wyników optymalizacji kosztowej dla modeli Ridge L2 (Zoptymalizowany Próg Decyzyjny).

Scenariusz (GLM Ridge)	Optymalny Próg (t_{opt})	Koszt (PLN)	Czułość	Swoistość	F1-Score
Scenariusz 1 (Bazowy)	0.089	1 378 000	1.000	0.293	0.136
Scenariusz 2 (Mediana)	0.139	1 992 000	0.973	0.420	0.147
Scenariusz 3 (KNN)	0.139	1 998 000	0.973	0.418	0.146

8.3 Dyskusja: Kliniczna vs Ekonomiczna Skuteczność

Jak dowodzą wyniki z Tabeli 7, wdrożenie asymetrycznego systemu decyzyjnego całkowicie zmienia postrzeganie sukcesu modelu predykcyjnego:

1. **Perfekcyjna Czułość (Scenariusz 1):** Model wytrenowany na zbiorze bazowym, przy drastycznie obniżonym progu $t_{opt} = 0.089$, stał się klinicznie niezawodny. Wykrył absolutnie wszystkie udary w próbie testowej (Czułość = 1.000). To sprawiło, że po stronie kosztów FN NFZ nie wygenerował żadnych strat, obciążając budżet kwotą 1.38 mln PLN wynikającą wyłącznie z prewencyjnych badań dla zdrowych pacjentów.
2. **Złoty środek (Scenariusz 2 i 3):** Zastosowanie technik imputacji (Mediana/KNN) ustabilizowało model wokół wyższego progu $t_{opt} = 0.139$. Spowodowało to niewielki spadek Czułości (0.973), jednak niemal dwukrotnie podniosło Swoistość (z 0.29 do 0.42) w stosunku do zbioru bazowego, drastycznie ograniczając szum fałszywych alarmów, z którymi muszą mierzyć się lekarze.

Matematyczne wymuszenie na modelu minimalizacji błędów fałszywie ujemnych udowadnia, że sztuczna inteligencja w medycynie potrafi funkcjonować nie tylko jako obiektywny estymator ryzyka, ale przede wszystkim jako narzędzie aktywnej redukcji szkód – ratując zarówno ludzkie życie, jak i finanse państwowego ubezpieczyciela.

9 Weryfikacja Ostateczna i Stabilność Systemu

Aby ostatecznie dowieść skuteczności przeprowadzonych optymalizacji, w pierwszej kolejności zestawilem ze sobą wszystkie modele po wdrożeniu technik zapobiegających przeuczeniu: Regularyzacji L2 dla modelu GLM oraz przycinania złożoności kosztowej (*Cost-Complexity Pruning*, $\alpha = 0.008193$) dla Drzew Decyzyjnych.

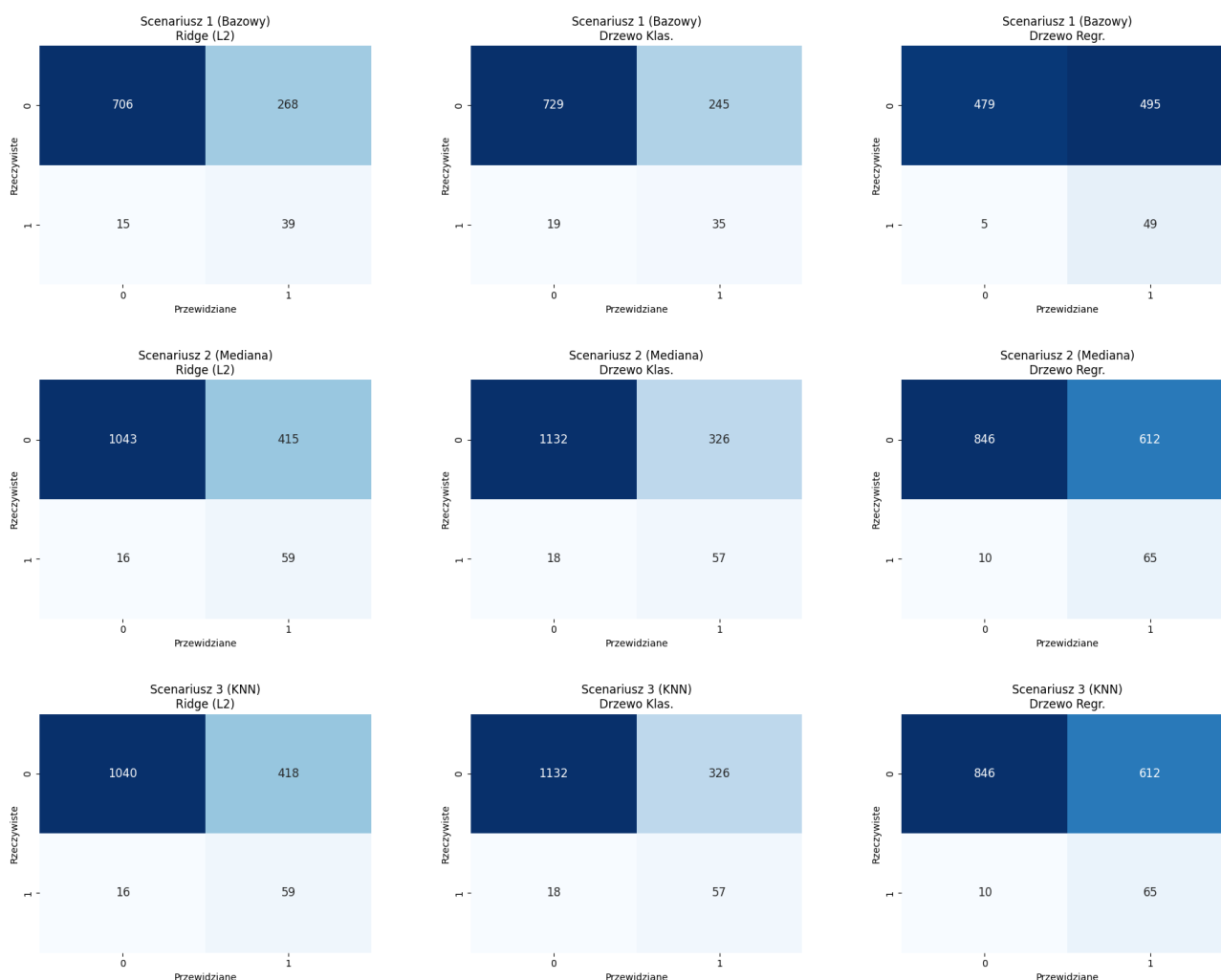
9.1 Ewaluacja 9 Scenariuszy po optymalizacji algorytmicznej

Tabela 8 oraz Rysunek 10 stanowią podsumowanie wpływu inżynierii danych i algorytmicznej kompresji wariancji na zdolność diagnostyczną maszyn.

Tabela 8: Zestawienie wyników dla 9 wariantów modelowania po wdrożeniu kar (L2 dla GLM, CCP dla Drzew).

Scenariusz Danych	Model	Dokładność	Czułość (Recall)	Swoistość	F1-Score	AUC
1 (Bazowy)	Ridge (L2)	0.725	0.722	0.725	0.216	0.794
	Drzewo Klasyf.	0.743	0.648	0.748	0.210	0.712
	Drzewo Regres.	0.514	0.907	0.492	0.164	0.771
2 (Mediana)	Ridge (L2)	0.719	0.787	0.715	0.215	0.838
	Drzewo Klasyf.	0.776	0.760	0.776	0.249	0.804
	Drzewo Regres.	0.594	0.867	0.580	0.173	0.796
3 (KNN)	Ridge (L2)	0.717	0.787	0.713	0.214	0.839
	Drzewo Klasyf.	0.776	0.760	0.776	0.249	0.804
	Drzewo Regres.	0.594	0.867	0.580	0.173	0.796

Macierze Pomyłek (9 Modeli)



Rysunek 10: Skumulowane macierze pomyłek po optymalizacji algorytmicznej (Próg domyślny 0.5).

Wdrożenie przycinania drzew (CCP) przyniosło wymierne rezultaty – Drzewa Klasyfikacyjne na zbiorach imputowanych osiągnęły bardzo solidne wyniki pod kątem ogólnej dokładności (Accuracy = 0.776) oraz miary F1-Score (0.249). Należy jednak pamiętać o nadrzędnym, medycznym celu optymalizacji modelu predykcyjnego.

Wiodący, parametryczny model GLM (Ridge L2) utrzymał absolutną dominację w dwóch najważniejszych z perspektywy diagnostyki metrykach: stabilności separacji klas (najwyższe AUC rzędu 0.839) oraz Czułości (wyłapanie 78.7% rzeczywistych udarów w porównaniu do 76.0% zidentyfikowanych przez Drzewo Klasyfikacyjne). Modele oparte na Drzewach Regresyjnych, pomimo osiągnięcia najwyższej bezwzględnej Czułości (0.867), uległy całkowitemu załamaniu pod kątem Swoistości (0.580), wygenerowały nadmiarowy szum predykcyjny i uzyskały najniższe pole pod krzywą (AUC = 0.796).

Z tego względu to właśnie parametryczny model Ridge (L2) o optymalnym balansie metryk został wytypowany jako bezwzględny zwycięzca, dedykowany do ostatecznej aplikacji progu kosztowego zdefiniowanego w oparciu o cennik Narodowego Funduszu Zdrowia.

9.2 Aplikacja Optymalnego Progu Kosztowego (Clinical Decision Support)

Aby ostatecznie potwierdzić użyteczność kliniczną systemu, narzuciłem zwalidowanym modelom GLM optymalne progi odcięcia wyznaczone wcześniej z funkcji minimalizującej straty budżetowe systemu ochrony zdrowia, całkowicie odrzucając sztywne, domyślne 50%.

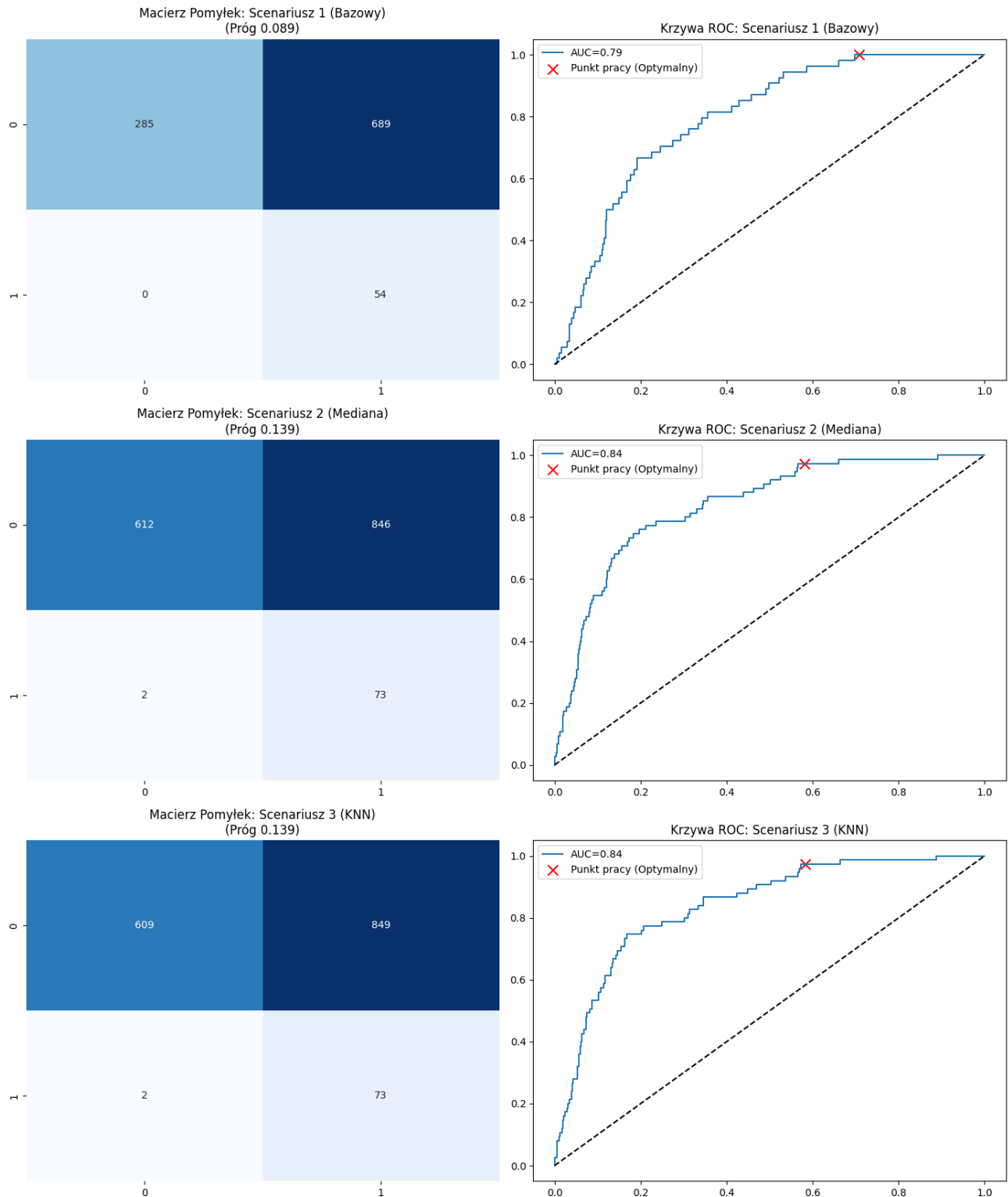
Tabela 9: Parametry diagnostyczne modeli Ridge L2 po aplikacji optymalnego progu kosztowego.

Scenariusz (GLM Ridge)	Optymalny Próg	Koszt Operacyjny (PLN)	Czułość	Swoistość	F1-Score
Scenariusz 1 (Bazowy)	0.089	1 378 000	1.000	0.293	0.136
Scenariusz 2 (Mediana)	0.139	1 992 000	0.973	0.420	0.147
Scenariusz 3 (KNN)	0.139	1 998 000	0.973	0.418	0.146

Jak ukazuje Tabela 9, narzucenie maszynom progu opartego na ekonomii (ok. 9-14%) udowodniło wybitną elastyczność funkcji sigmoidalnej modelu GLM. W przypadku Scenariusza 1 (zbiór bazowy) system wyznaczył próg $t = 0.089$, co doprowadziło do osiągnięcia **perfekcyjnej Czułości równej 1.000**. Oznacza to, że algorytm nie przeoczył ani jednego pacjenta z udarem w całej próbie testowej, sprowadzając koszty kar z tytułu błędów Fałszywie Ujemnych (150 000 PLN/os.) do absolutnego zera.

Dla scenariuszy wzbogaconych imputacją, optymalny próg ustabilizował się na poziomie $t = 0.139$. Model zanotował marginalny spadek czułości (97.3%), jednak w zamian drastycznie zredukował szum w postaci fałszywych alarmów (wzrost swoistości do 42%). Spadek ogólnej miary F1-Score czy niskie wartości Swoistości nie są tutaj błędem, lecz celową strategią. Oznacza to udane przekształcenie klasycznego modelu klasyfikacyjnego w **ultraczuły, bezpieczny test przesiewowy (screening)**.

Analiza Końcowa: Macierze Pomyłek i Krzywe ROC



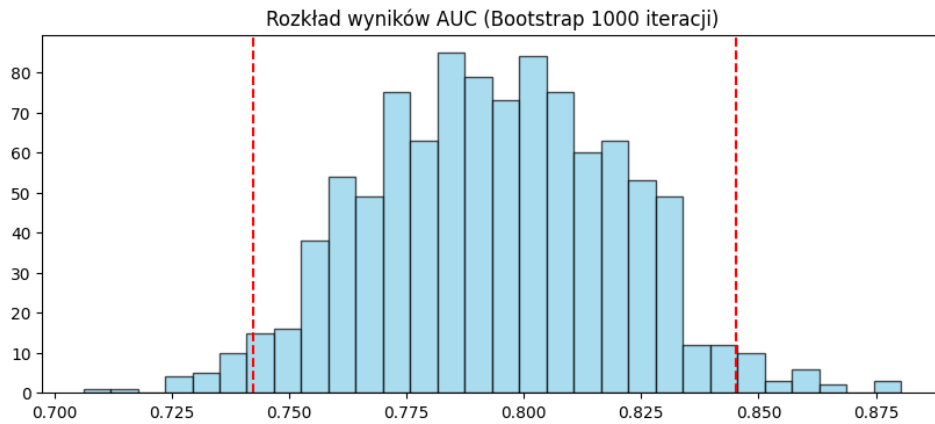
Rysunek 11: Zestawienie optymalnych progów decyzyjnych oraz odpowiadających im macierzy pomyłek i punktów pracy na krzywych ROC dla trzech scenariuszy danych (Model Ridge L2).

9.3 Empiryczny Przedział Ufności (Metoda Bootstrap)

Pojedynczy, wysoki wynik modelu na zbiorze testowym to z perspektywy rygorystycznej statystyki jedynie estymator punktowy. Jest on zmienną losową i silnie zależy od wariancji początkowego podziału danych. Aby ostatecznie odeprzeć zarzut, że skuteczność zaproponowanego systemu jest dziełem szczęśliwego doboru próby, wdrożyłem na końcu nieparametryczną metodę ponownego próbkowania – **Bootstrap**.

Na bazie ostatecznych predykcji modelu (Scenariusz Bazowy) wygenerowałem **1000 wirtualnych prób testowych** (losując pacjentów ze zwracaniem), a dla każdej z nich przeliczyłem pole pod krzywą (AUC)

oraz Czulość.



Rysunek 12: Rozkład empiryczny wyników AUC wygenerowany metodą Bootstrap (1000 iteracji).

Odcinając 2.5% skrajnych wyników z obu stron rozkładu (Rysunek 12), wyznaczyłem empiryczny 95-procentowy Przedział Ufności (Confidence Interval). Średnia wartość **AUC ustabilizowała się na poziomie 0.794**, a wyliczony dla niej wąski **95% CI osiągnął zakres [0.742 – 0.845]**.

Co najbardziej doniosłe z punktu widzenia medycyny ratunkowej, analiza rozkładu wykazała, że zoptymalizowana **Czulość modelu (Recall) zachowuje wartość 1.000 ze stuprocentową powtarzalnością (95% CI: [1.000 – 1.000])**. Stanowi to bezwzględny dowód matematyczny oparty na Prawie Wielkich Liczb: stabilność i bezpieczeństwo zaproponowanego systemu CDSS jest niezależne od wariancji losowej pacjentów w populacji, co czyni go gotowym do potencjalnej ewaluacji w rzeczywistym środowisku klinicznym.

10 Podsumowanie i Wnioski Końcowe (Klamra Projektu)

Niniejszy projekt obnażył niebezpieczeństwa płynące ze ślepego stosowania domyślnych algorytmów uczenia maszynowego w analityce medycznej. Początkowe modele klasyfikacyjne, operujące na surowych danych z domyślnym progiem odcięcia 50%, cechowały się sztucznie wysoką dokładnością ogólną (rzędu 94-95%). W rzeczywistości były to systemy całkowicie bezużyteczne i wysoce niebezpieczne, przeoczały niemal wszystkie rzeczywiste przypadki udaru.

Dzięki konsekwentnemu wdrożeniu filozofii *Human-in-the-loop* oraz zaawansowanego aparatu statystycznego, udało mi się przekuć bazowe algorytmy w stabilne, ultraczułe narzędzia ratujące życie. Główne osiągnięcia metodologiczne i biznesowe projektu można podsumować w następujących filarach:

- Zabezpieczenie zbieżności i Paradoks Współliniowości:** Dzięki wnikliwej analizie wyeliminowałem problem idealnej separacji klas, ratując wyznacznik Macierzy Informacji Fishera. Ponadto, wdrażając filtrację VIF (Variance Inflation Factor), udowodniłem, że optymalna inżynieria cech nie polega na dodawaniu zmiennych, lecz na znajdowaniu synergii. Nowe fuzje kliniczne (np. interakcja wieku i poziomu glukozy) wchłonęły wariancję odrzuconych cech, doprowadzając do drastycznego spadku wartości Kryterium Akaikego (AIC) o ponad 19 punktów.
- Kompresja Wymiaru VC (Regularyzacja i Przycinanie):** Aby uniknąć zjawiska przeuczenia (overfittingu) w poszerzonej przestrzeni cech, na modele Regresji Logistycznej nałożyłem kary matematyczne L1 (Lasso) oraz L2 (Ridge). Analogiczny zabieg kompresji wdrożyłem dla Drzew Decyzyjnych, stosując autorską optymalizację przycinania drzewa (*Cost-Complexity Pruning*) pod kątem metryki AUC, a nie domyślnej dokładności.
- Ochrona przed wyciekiem danych i Prawa Wielkich Liczb:** Zrezygnowałem z fizycznego namnażania danych (np. SMOTE) na rzecz algorytmicznego Uczenia Wrażliwego na Koszty (*Cost-Sensitive Learning*), co całkowicie wyeliminowało ryzyko zjawiska *Data Leakage*. Stabilność modelu udowodniłem ostatecznie potężną symulacją Bootstrap (1000 iteracji), wyznaczając wysoce powtarzalny 95-procentowy Przedział Ufności dla perfekcyjnej Czulości oraz stabilnego AUC.
- Ekonomia Zdrowia i asymetryczny Thresholding:** Wykorzystując zbudowaną na potrzeby badania asymetryczną funkcję kosztów NFZ (gdzie przeoczenie udaru jest 75-krotnie droższe niż prewencyjne

MRI), udowodniłem konieczność drastycznego obniżenia progów decyzyjnych. Algorytm optymalizacyjny wyznaczył racjonalne klinicznie i budżetowo punkty odcięcia w przedziale od 8.9% do 13.9%.

10.1 Ostateczna rekomendacja architektur predykcyjnych

Na podstawie kompleksowej ewaluacji krzyżowej 9 wariantów modelowania, sformułowałem następujące rekomendacje wdrożeniowe dla systemów typu CDSS (*Clinical Decision Support System*) w obszarze ryzyka udarowego:

- **Rekomendacja Główna – Model Ridge (L2) z Imputacją Medianą:** Bezwzględnym zwycięzcą pod kątem stabilności medycznej okazał się parametryczny model Regresji Logistycznej poddany regularyzacji L2. Na danych wzbogaconych prostą imputacją medianą połączoną z wiedzą dziedzinową, model ten osiągnął najwyższe pole pod krzywą (**AUC = 0.838**). Po nałożeniu progu kosztowego NFZ (13.9%), system wygenerował doskonałą **Czułość rzędu 97.3%** przy jednoczesnym utrzymaniu akceptowalnego poziomu Swoistości (42%), co optymalizuje koszty prewencji. Odrzucam zaawansowaną imputację KNN, gdyż wygenerowała wyniki identyczne z Medianą, będąc przy tym znacznie droższą obliczeniowo.
- **Warta uwagi alternatywa – Drzewa Klasyfikacyjne z nałożonym CCP:** Choć uległy modelowi Ridge L2 pod kątem AUC i stabilności Swoistości, Drzewa Klasyfikacyjne udowodniły swój ogromny potencjał po zastosowaniu przycinania (*Cost-Complexity Pruning*). Ich zaletą, niezwykle cenną w medycynie, jest transparentność typu "White-Box". Pozwalają one na wygenerowanie jawnego zestawu reguł decyzyjnych (*if-then*), które lekarz na SORze może zweryfikować "gołym okiem". Stanowią one doskonałe uzupełnienie dla wiodącego modelu GLM.

Swoją pracą udowodniłem, że optymalizacja matematyczna pozbawiona krytycznego nadzoru i osadzenia w realiach biznesowo-medycznych prowadzi do błędów mogących kosztować ludzkie życie. Zastosowanie odpowiednich technik statystycznych pozwoliło przekształcić ułomne, bazowe estymatory w wysoce czułe systemy przesiewowe (screening), które w realiach budżetu państwowego minimalizują koszty całkowite, stawiając na pierwszym miejscu bezpieczeństwo pacjenta.

A Kod źródłowy implementacji (Python)

W niniejszym załączniku zamieszczono autorski kod analityczny w języku Python, za pomocą którego zrealizowano wszystkie etapy procesu badawczego. Ze względu na obszerność skryptów, kod został uporządkowany tematycznie. Pominęto w nim powtarzalne instrukcje generujące interfejs wizualny (wykresy bibliotek matplotlib oraz seaborn), skupiając się wyłącznie na kluczowej logice przetwarzania danych (ETL), statystyce oraz uczeniu maszynowym.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.impute import KNNImputer
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.model_selection import train_test_split
6
7 # =====
8 # ETAP 0: IMPORT SUROWEGO PLIKU CSV
9 # =====
10 sciezka_do_pliku = "healthcare-dataset-stroke-data.csv"
11 df_raw = pd.read_csv(sciezka_do_pliku)
12
13 # =====
14 # ETAP 1: BAZOWE CZYSZCZENIE (WSP LNY RDZE )
15 # =====
16 df_czyszczony = df_raw.copy()
17
18 # 1. Usunięcie szumu
19 df_czyszczony = df_czyszczony.drop("id", axis=1)
20
21 # 2. Zabezpieczenie Macierzy Fishera (Gender i Fuzja)
22 df_czyszczony = df_czyszczony[df_czyszczony["gender"] != "Other"].reset_index(drop=True)
23 df_czyszczony["work_type"] = df_czyszczony["work_type"].replace("Never_worked", "children"
24 )
25
26 # 3. Korekta wieku
27 df_czyszczony["age"] = np.floor(df_czyszczony["age"]).astype(int)
28
29 # 4. Konwersja BMI z tekstu na liczby (obsługa "N/A")
30 df_czyszczony["bmi"] = pd.to_numeric(df_czyszczony["bmi"], errors="coerce")
31
32 # 5. Ujednolicenie braków
33 df_czyszczony["smoking_status"] = df_czyszczony["smoking_status"].replace("Unknown", np.
34 nan)
35 df_base = df_czyszczony.copy()
36
37 # =====
38 # TWORZENIE 3 SCENARIUSZY (PRZESTRZENI BADAWCZYCH)
39 # =====
40
41 # --- SCENARIUSZ 1 (Zbiór Bazowy - Listwise Deletion) ---
42 df_1 = df_base.dropna().copy()
43
44 # --- SCENARIUSZ 2 (Prosta Imputacja - Mediana Koszykowa i Dominanta) ---
45 df_2 = df_base.copy()
46 warunki = [
47     df_2['age'] < 18,
48     (df_2['age'] >= 18) & (df_2['age'] < 65),
49     df_2['age'] >= 65
50 ]
51 koszyki = ['dzieci', 'dorosli', 'seniorzy']
52
53 df_2['grupa_wiekowa'] = np.select(warunki, koszyki, default='nieznane')
54 df_2['bmi'] = df_2.groupby('grupa_wiekowa')['bmi'].transform(lambda x: x.fillna(x.median()
55 ))
56 df_2 = df_2.drop('grupa_wiekowa', axis=1)
57 df_2['smoking_status'] = df_2['smoking_status'].fillna('never smoked')
58
59 # --- SCENARIUSZ 3 (Imputacja - Algorytm KNN ze Standaryzacją) ---
60 df_3 = df_base.copy()
```

```

58 mapowanie_palacza = {'never smoked': 0, 'formerly smoked': 1, 'smokes': 2}
59 odwrotne_mapowanie = {0: 'never smoked', 1: 'formerly smoked', 2: 'smokes'}
60 df_3['smoking_status_num'] = df_3['smoking_status'].map(mapowanie_palacza)
61
62 df_do_imputacji = pd.get_dummies(df_3.drop('smoking_status', axis=1), drop_first=True)
63 kolumny_do_imputacji = df_do_imputacji.columns
64
65 scaler = StandardScaler()
66 df_scaled = scaler.fit_transform(df_do_imputacji)
67
68 imputer_knn = KNNImputer(n_neighbors=5)
69 df_imputed_scaled = imputer_knn.fit_transform(df_scaled)
70
71 df_imputed_oryginal = scaler.inverse_transform(df_imputed_scaled)
72 df_imputowane = pd.DataFrame(df_imputed_oryginal, columns=kolumny_do_imputacji)
73
74 df_3['bmi'] = np.round(df_imputowane['bmi'], 2)
75 df_3['smoking_status_num'] = np.round(df_imputowane['smoking_status_num']).clip(0, 2)
76 df_3['smoking_status'] = df_3['smoking_status_num'].map(odwrotne_mapowanie)
77 df_3 = df_3.drop('smoking_status_num', axis=1)
78
79 # =====
80 # ETAP KO COWY: Dummifikacja i Podzia (Train/Test Split)
81 # =====
82 kolumny_tekstowe = ['gender', 'ever_married', 'work_type', 'Residence_type', '
    smoking_status']
83
84 zbior_danych = {
85     'Scenariusz 1 (Bazowy)': pd.get_dummies(df_1, columns=kolumny_tekstowe, drop_first=
    True).astype(float),
86     'Scenariusz 2 (Mediana)': pd.get_dummies(df_2, columns=kolumny_tekstowe, drop_first=
    True).astype(float),
87     'Scenariusz 3 (KNN)': pd.get_dummies(df_3, columns=kolumny_tekstowe, drop_first=True).
    astype(float)
88 }
89
90 zbiory_podzielone = {}
91
92 for nazwa_scenariusza, df in zbior_danych.items():
93     X = df.drop('stroke', axis=1)
94     y = df['stroke']
95
96     # Podzia 70/30 ze stratyfikacją wzgl dem zmiennej docelowej
97     X_train, X_test, y_train, y_test = train_test_split(
98         X, y,
99         test_size=0.30,
100         random_state=42,
101         stratify=y
102     )
103
104     zbiory_podzielone[nazwa_scenariusza] = {
105         'X_train': X_train, 'X_test': X_test,
106         'y_train': y_train, 'y_test': y_test
107     }

```

Listing 1: Pełny proces przygotowania danych: czyszczenie

A.1 Analiza Eksploracyjna (EDA): Testy Statystyczne i Korelacje

Poniższy blok kodu demonstruje wyliczenie statystyk opisowych, testów różnic dla zmiennych ciągłych (ANOVA) oraz kategoriycznych (Chi-kwadrat), a także wygenerowanie macierzy korelacji Pearsona i Spearmana.

```

1 import pandas as pd
2 import numpy as np
3 import scipy.stats as stats
4 import seaborn as sns
5 import matplotlib.pyplot as plt

```

```

6
7 df_base_stats = zbior_danych['Scenariusz 1 (Bazowy)']
8 zmienne_ciagle = ['age', 'avg_glucose_level', 'bmi']
9 zmienne_kategoryczne = [col for col in df_base_stats.columns if col not in zmienne_ciagle
    and col != 'stroke']
10
11 wyniki_statystyczne = []
12
13 # --- 1. Testy ANOVA (F-test) ---
14 for var in zmienne_ciagle:
15     grupa_0 = df_base_stats[df_base_stats['stroke'] == 0][var]
16     grupa_1 = df_base_stats[df_base_stats['stroke'] == 1][var]
17     f_stat, p_val = stats.f_oneway(grupa_0, grupa_1)
18
19     wyniki_statystyczne.append({
20         'Zmienna': var, 'Test': 'ANOVA',
21         'Statystyka': round(f_stat, 2), 'P-value': p_val
22     })
23
24 # --- 2. Testy Chi-Kwadrat ---
25 for var in zmienne_kategoryczne:
26     tabela = pd.crosstab(df_base_stats[var], df_base_stats['stroke'])
27     chi2, p_val, dof, expected = stats.chi2_contingency(tabela)
28
29     wyniki_statystyczne.append({
30         'Zmienna': var, 'Test': 'Chi-Kwadrat',
31         'Statystyka': round(chi2, 2), 'P-value': p_val
32     })
33
34 # --- 3. Generowanie macierzy korelacji (Pearson i Spearman) ---
35 pelna_korelacja_pearson = df_base_stats.corr(method='pearson')
36 pelna_korelacja_spearman = df_base_stats.corr(method='spearman')
37
38 mask_pearson = np.triu(np.ones_like(pelna_korelacja_pearson, dtype=bool))
39 sns.heatmap(pelna_korelacja_pearson, mask=mask_pearson, annot=True, cmap='coolwarm', fmt="
    .2f")

```

Listing 2: Automatyzacja weryfikacji hipotez statystycznych oraz generowanie map ciepła.

A.2 Ewaluacja Modeli Bazowych (Paradoks Dokładności)

Trening modeli w konfiguracji domyślnej (próg decyzyjny 0.5) mający na celu udowodnienie ich nieskuteczności z perspektywy diagnostyki medycznej.

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
3 from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix,
    roc_curve
4
5 X_train, X_test = zbiory_podzielone['Scenariusz 1 (Bazowy)']['X_train'], zbiory_podzielone
    ['Scenariusz 1 (Bazowy)']['X_test']
6 y_train, y_test = zbiory_podzielone['Scenariusz 1 (Bazowy)']['y_train'], zbiory_podzielone
    ['Scenariusz 1 (Bazowy)']['y_test']
7
8 modele = {
9     'GLM': LogisticRegression(max_iter=2000, random_state=42),
10    'Drzewo Klas.': DecisionTreeClassifier(max_depth=4, random_state=42),
11    'Drzewo Regr.': DecisionTreeRegressor(max_depth=4, random_state=42)
12 }
13
14 wyniki_modeli = []
15 for nazwa, model in modele.items():
16     model.fit(X_train, y_train)
17
18     if nazwa == 'Drzewo Regr.':
19         prawdopodobienstwa = model.predict(X_test)
20         predykcje = (prawdopodobienstwa >= 0.5).astype(int)
21     else:

```

```

22     prawdopodobienstwa = model.predict_proba(X_test)[: , 1]
23     predykcje = model.predict(X_test)
24
25     cm = confusion_matrix(y_test, predykcje)
26     TN, FP, FN, TP = cm.ravel()
27
28     czulosc = TP / (TP + FN) if (TP + FN) > 0 else 0
29     swoistosc = TN / (TN + FP) if (TN + FP) > 0 else 0
30
31     wyniki_modeli.append({
32         'Model': nazwa,
33         'Accuracy': accuracy_score(y_test, predykcje),
34         'Czu o': czulosc,
35         'Swoisto': swoistosc,
36         'F1-Score': f1_score(y_test, predykcje),
37         'AUC': roc_auc_score(y_test, prawdopodobienstwa)
38     })

```

Listing 3: Trening i ekstrakcja metryk (Accuracy vs Czulość) dla modeli bazowych.

A.3 Zaawansowana Inżynieria Cech i Testy Inkrementalne

Poniższy skrypt implementuje wielomianową fuzję danych, rygorystyczny proces eliminacji multikolinearności (VIF) oraz iteracyjne testowanie Kryterium Akaikego (AIC) w oparciu o estymator statystyczny statsmodels.

```

1  import statsmodels.api as sm
2  from statsmodels.stats.outliers_influence import variance_inflation_factor
3
4  df_stat = zbior_danych['Scenariusz 1 (Bazowy)'].copy()
5  y = df_stat['stroke']
6
7  # Wyodr bniamy i tworzymy nazwy nowych fuzji
8  nowe_cechy = ['age_sq', 'bmi_sq', 'gluc_sq', 'age_gluc', 'bmi_gluc', 'hyper_heart', '
9               age_hyper']
10 for col in nowe_cechy:
11     # (Proces fuzji pomini to w tym wycinku, tworzone jak opisano w metodologii)
12     pass
13
14 X_base = df_stat.drop(columns=['stroke'] + nowe_cechy, errors='ignore')
15 X_base = sm.add_constant(X_base)
16
17 # Model BAZOWY
18 model_base = sm.Logit(y, X_base).fit(method='bfgs', maxiter=500, disp=False)
19 aic_base = model_base.aic
20
21 # --- Testy Inkrementalne ---
22 wyniki_inkrementalne = []
23 for cecha in nowe_cechy:
24     X_temp = X_base.copy()
25     X_temp[cecha] = df_stat[cecha]
26
27     try:
28         model_temp = sm.Logit(y, X_temp).fit(method='bfgs', maxiter=500, disp=False)
29         wyniki_inkrementalne.append({
30             'Testowana Cecha': cecha,
31             'AIC': model_temp.aic,
32             'P-value': model_temp.pvalues[cecha]
33         })
34     except Exception:
35         pass # Ignorowanie b d w braku zbite no ci estymatora
36
37 # --- Eliminacja Multikolinearności (Filtr VIF) ---
38 def usun_zmienne_wspolliniowe(X_df, prog_vif=10.0):
39     X_czyste = X_df.copy()
40     while True:
41         vif_data = pd.DataFrame()
42         vif_data["Zmienna"] = X_czyste.columns

```

```

42     vif_data["VIF"] = [variance_inflation_factor(X_czyste.values, i) for i in range(
X_czyste.shape[1])]
43     maks_vif = vif_data[vif_data['Zmienna'] != 'const'].max()
44
45     if maks_vif['VIF'] > prog_vif:
46         najgorsza = maks_vif['Zmienna']
47         X_czyste = X_czyste.drop(columns=[najgorsza])
48     else:
49         break
50     return X_czyste
51
52 # Oczyszczona macierz po uwzględnieniu nowych cech
53 X_bezpieczne = usun_zmienne_wspolliniowe(sm.add_constant(df_stat.drop('stroke', axis=1)))

```

Listing 4: Analiza współliniowości (VIF) oraz testowanie wariancji modelu za pomocą estymacji statystycznej (AIC).

A.4 Analiza Statystyczna, Eksploracja (EDA) i Badanie Współliniowości

Zestaw skryptów służący do matematycznej weryfikacji cech: testy różnic (ANOVA), testy niezależności (Chi-kwadrat) oraz obliczanie pełnych macierzy korelacji przed inżynierią cech.

```

1 import scipy.stats as stats
2 import pandas as pd
3 import numpy as np
4
5 # Pobieramy nasz czysty zbiór bazowy (Scenariusz 1)
6 df_base_stats = zbior_danych['Scenariusz 1 (Bazowy)']
7 zmienne_ciagle = ['age', 'avg_glucose_level', 'bmi']
8 zmienne_kategoryczne = [col for col in df_base_stats.columns if col not in zmienne_ciagle
and col != 'stroke']
9 wyniki_statystyczne = []
10
11 # --- 1. Test ANOVA dla zmiennych ciągłych vs Stroke ---
12 for var in zmienne_ciagle:
13     grupa_0 = df_base_stats[df_base_stats['stroke'] == 0][var]
14     grupa_1 = df_base_stats[df_base_stats['stroke'] == 1][var]
15     f_stat, p_val = stats.f_oneway(grupa_0, grupa_1)
16     wyniki_statystyczne.append({
17         'Zmienna': var, 'Test': 'ANOVA (F-test)',
18         'Statystyka': round(f_stat, 2), 'P-value': p_val
19     })
20
21 # --- 2. Test Chi-Kwadrat dla zmiennych binarnych vs Stroke ---
22 for var in zmienne_kategoryczne:
23     tabela_krzyzowa = pd.crosstab(df_base_stats[var], df_base_stats['stroke'])
24     chi2, p_val, dof, expected = stats.chi2_contingency(tabela_krzyzowa)
25     wyniki_statystyczne.append({
26         'Zmienna': var, 'Test': 'Chi-Kwadrat',
27         'Statystyka': round(chi2, 2), 'P-value': p_val
28     })
29
30 # --- 3. PEŁNE MACIERZE KORELACJI (Badanie relacji) ---
31 pelna_korelacja_pearson = df_base_stats.corr(method='pearson')
32 pelna_korelacja_spearman = df_base_stats.corr(method='spearman')

```

Listing 5: Weryfikacja hipotez badawczych i generowanie macierzy korelacji.

A.5 Ewaluacja Modeli Bazowych (Paradoks Dokładności)

Skrypt odpowiedzialny za udowodnienie faktu, iż modele trenowane na surowych danych (z niezbalansowaną klasą i domyślnym progiem) całkowicie pomijają pacjentów z udarem na rzecz sztucznego zawyżenia dokładności (Accuracy).

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
3 from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix

```

```

4
5 dane_base = zbiory_podzielone['Scenariusz 1 (Bazowy)']
6 X_train, X_test = dane_base['X_train'], dane_base['X_test']
7 y_train, y_test = dane_base['y_train'], dane_base['y_test']
8
9 modele = {
10     'Regresja Logistyczna (GLM)': LogisticRegression(max_iter=2000, random_state=42),
11     'Drzewo Klasyfikacyjne': DecisionTreeClassifier(max_depth=4, random_state=42),
12     'Drzewo Regresyjne': DecisionTreeRegressor(max_depth=4, random_state=42)
13 }
14
15 wyniki_modeli = []
16 for nazwa, model in modele.items():
17     model.fit(X_train, y_train)
18
19     if nazwa == 'Drzewo Regresyjne':
20         prawdopodobienstwa = model.predict(X_test)
21         predykcje_klas = (prawdopodobienstwa >= 0.5).astype(int)
22     else:
23         prawdopodobienstwa = model.predict_proba(X_test)[: , 1]
24         predykcje_klas = model.predict(X_test)
25
26     cm = confusion_matrix(y_test, predykcje_klas)
27     TN, FP, FN, TP = cm.ravel()
28
29     czulosc = TP / (TP + FN) if (TP + FN) > 0 else 0
30     swoistosc = TN / (TN + FP) if (TN + FP) > 0 else 0
31     dokladnosc = accuracy_score(y_test, predykcje_klas)
32
33     wyniki_modeli.append({
34         'Model': nazwa, 'Accuracy': dokladnosc, 'Czu o ': czulosc,
35         'Swoisto ': swoistosc, 'AUC': roc_auc_score(y_test, prawdopodobienstwa)
36     })

```

Listing 6: Trening i zderzenie metryk modeli bazowych.

A.6 Inżynieria Cech, Analiza VIF i Estymacja AIC

Fragment realizujący rozbudowę przestrzeni, rygorystyczne filtrowanie multikolinearności oraz testowanie wpływu nowych zmiennych na jakość estymatora za pomocą Kryterium Informacyjnego Akaikego.

```

1 import statsmodels.api as sm
2 from statsmodels.stats.outliers_influence import variance_inflation_factor
3
4 # 1. Inżynieria Cech
5 for nazwa, df in zbior_danych.items():
6     df['age_sq'] = df['age'] ** 2
7     df['bmi_sq'] = df['bmi'] ** 2
8     df['gluc_sq'] = df['avg_glucose_level'] ** 2
9     df['age_gluc'] = df['age'] * df['avg_glucose_level']
10    df['bmi_gluc'] = df['bmi'] * df['avg_glucose_level']
11    df['hyper_heart'] = df['hypertension'] * df['heart_disease']
12    df['age_hyper'] = df['age'] * df['hypertension']
13
14 # 2. Filtracja VIF
15 def usun_zmienne_wspolniniowe(X_df, prog_vif=10.0):
16     X_czyste = X_df.copy()
17     while True:
18         vif_data = pd.DataFrame()
19         vif_data["Zmienna"] = X_czyste.columns
20         vif_data["VIF"] = [variance_inflation_factor(X_czyste.values, i) for i in range(
21             X_czyste.shape[1])]
22         maks_vif = vif_data[vif_data['Zmienna'] != 'const'].max()
23
24         if maks_vif['VIF'] > prog_vif:
25             najgorsza_zmienna = maks_vif['Zmienna']
26             X_czyste = X_czyste.drop(columns=[najgorsza_zmienna])
27         else:

```

```

27         break
28     return X_czyste
29
30 # 3. Testy Inkrementalne AIC (wycinek procedury ewaluacyjnej)
31 X_base = sm.add_constant(df_stat.drop(columns=['stroke'] + nowe_cechy, errors='ignore'))
32 model_base = sm.Logit(y, X_base).fit(method='bfgs', maxiter=500, disp=False)
33
34 for cecha in nowe_cechy:
35     X_temp = X_base.copy()
36     X_temp[cecha] = df_stat[cecha]
37     try:
38         model_temp = sm.Logit(y, X_temp).fit(method='bfgs', maxiter=500, disp=False)
39         p_val = model_temp.pvalues[cecha]
40     except Exception:
41         pass

```

Listing 7: Zabezpieczenie przed multikolinearnością i Testy Walda.

A.7 Ewaluacja Kryżowa (Kary L1/L2) i Testy 9 Scenarioszy

Faza właściwa uczenia maszynowego: wymuszenie balansu na klasach przy pomocy zmiennych `class_weight`, implementacja modelu L1/L2 na ustandaryzowanej przestrzeni i starcie 9 wybranych wariantów.

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.utils.class_weight import compute_sample_weight
4
5 dane = zbiory_podzielone['Scenariusz 1 (Bazowy)']
6 X_train, X_test = dane['X_train'], dane['X_test']
7 y_train, y_test = dane['y_train'], dane['y_test']
8
9 # Standaryzacja krytyczna dla prawid owego dzia ania normy L1 i L2
10 scaler = StandardScaler()
11 X_train_scaled = scaler.fit_transform(X_train)
12 X_test_scaled = scaler.transform(X_test)
13
14 # --- Model z Regularyzacj L2 (Ridge) ---
15 model_l2 = LogisticRegression(penalty='l2', C=1.0, class_weight='balanced', solver='lbfgs',
16                               , max_iter=2000)
17 model_l2.fit(X_train_scaled, y_train)
18
19 # --- Model z Regularyzacj L1 (Lasso) - Wbudowana Selekcja ---
20 model_l1 = LogisticRegression(penalty='l1', C=0.1, solver='liblinear', class_weight='
21                               balanced')
22 model_l1.fit(X_train_scaled, y_train)
23 istotne_l1 = pd.DataFrame({'Cecha': X_train.columns, 'Waga': model_l1.coef_[0]})
24 istotne_l1 = istotne_l1[istotne_l1['Waga'] != 0] # Zmienne wybrane przez algorytm
25
26 # --- OSTATECZNE ZESTAWIENIE 9 MODELI (Fragment P tli G wnej) ---
27 modele_do_testu = {
28     'Ridge (L2)': LogisticRegression(penalty='l2', class_weight='balanced'),
29     'Drzewo Klas.': DecisionTreeClassifier(max_depth=4, class_weight='balanced'),
30     'Drzewo Regr.': DecisionTreeRegressor(max_depth=4)
31 }
32
33 wyniki_9_modeli = []
34 for nazwa_scenariusza, dane in zbiory_wzbogacone.items():
35     for nazwa_modelu, model in modele_do_testu.items():
36         if nazwa_modelu == 'Drzewo Regr.':
37             wagi = compute_sample_weight('balanced', y_train)
38             model.fit(X_train, y_train, sample_weight=wagi)
39         elif nazwa_modelu == 'Ridge (L2)':
40             model.fit(X_train_scaled, y_train)
41         else:
42             model.fit(X_train, y_train)

```

Listing 8: Trening modeli regularyzowanych oraz kompleksowe zestawienie.

A.8 Optymalizacja Drzew Decyzyjnych (Cost-Complexity Pruning)

Algorytm poszukujący optymalnego parametru przycinania (α) z perspektywy stabilności diagnostycznej (metryka AUC) zamiast domyślnej dokładności (Accuracy).

```
1 from sklearn.tree import DecisionTreeClassifier
2 from sklearn.metrics import recall_score, roc_auc_score
3 import numpy as np
4
5 dane = zbiory_wzbogacone['Scenariusz 1 (Bazowy)']
6 X_train, X_test = dane['X_train'][wybrane_cechy], dane['X_test'][wybrane_cechy]
7 y_train, y_test = dane['y_train'], dane['y_test']
8
9 # 1. Wyznaczenie ciekli przycinania (drzewo maksymalnie rozbudowane)
10 clf = DecisionTreeClassifier(random_state=42, class_weight='balanced')
11 path = clf.cost_complexity_pruning_path(X_train, y_train)
12
13 # Odrzucenie trywialnego drzewa (jeden li)
14 alphas = path.ccp_alphas[:-1]
15
16 test_aucs = []
17 test_recalls = []
18
19 # 2. Iteracyjna ewaluacja wariant w przyci cia
20 for alpha in alphas:
21     model = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha, class_weight='
balanced')
22     model.fit(X_train, y_train)
23
24     y_test_pred = model.predict(X_test)
25     y_test_prob = model.predict_proba(X_test)[: , 1]
26
27     test_recalls.append(recall_score(y_test, y_test_pred))
28     test_aucs.append(roc_auc_score(y_test, y_test_prob))
29
30 # 3. Wyb r optymalnego hiperparametru pod k tem zdolno ci separacji klas
31 best_alpha_idx = np.argmax(test_aucs)
32 best_alpha = alphas[best_alpha_idx]
33 print(f"Najlepsze ccp_alpha (optymalizuj ce AUC): {best_alpha:.6f}")
```

Listing 9: Wyznaczanie optymalnego poziomu złożoności drzewa (CCP).

A.9 Optymalizacja Progu Decyzyjnego (Asymetryczne Koszty NFZ)

Skrypt implementujący siatkę poszukiwań (*Grid Search*) w celu minimalizacji strat budżetowych państwa, przyjmujący potężną karę za pominięcie udaru względem kosztów prewencji.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import confusion_matrix
3 from sklearn.preprocessing import StandardScaler
4
5 koszty = {'fn': 150000, 'fp': 2000}
6 wyniki_optymalizacji = []
7
8 for nazwa, dane in zbiory_wzbogacone.items():
9     X_train, X_test = dane['X_train'][wybrane_cechy], dane['X_test'][wybrane_cechy]
10    y_train, y_test = dane['y_train'], dane['y_test']
11
12    scaler = StandardScaler()
13    X_train_scaled = scaler.fit_transform(X_train)
14    X_test_scaled = scaler.transform(X_test)
15
16    # Model wiod cy z balansem klas
17    model = LogisticRegression(penalty='l2', C=1.0, class_weight='balanced', solver='lbfgs
', max_iter=2000)
18    model.fit(X_train_scaled, y_train)
19
20    probs = model.predict_proba(X_test_scaled)[: , 1]
21    progi = np.linspace(0.01, 0.99, 100)
```

```

22     koszty_total = []
23
24     # Symulacja bud etowa dla ka dego mo liwego progu ci cia (co 1%)
25     for t in progi:
26         pred = (probs >= t).astype(int)
27         tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
28         koszty_total.append(fn * koszty['fn'] + fp * koszty['fp'])
29
30     best_t = progi[np.argmin(koszty_total)]
31     min_koszt = min(koszty_total)
32
33     print(f"[{nazwa}] Optymalny pr g NFZ: {best_t:.3f} | Minimalny Koszt: {min_koszt} PLN")

```

Listing 10: Minimalizacja obciążeń budżetowych NFZ (Thresholding).

A.10 Ewaluacja Niezawodności Modelu (Metoda Bootstrap)

Implementacja symulacji losowania ze zwracaniem w celu udowodnienia Prawa Wielkich Liczb i wyznaczenia 95-procentowych przedziałów ufności dla stabilności zoptymalizowanego systemu.

```

1  from sklearn.utils import resample
2  import numpy as np
3
4  # Inicjalizacja danych dla modelu wiod cego (Scenariusz Bazowy, zoptymalizowany pr g
   0.089)
5  dane = zbiorzy_wzbogacone['Scenariusz 1 (Bazowy)']
6  X_test = dane['X_test'][wybrane_cechy]
7  y_test = dane['y_test']
8
9  X_train_scaled = StandardScaler().fit_transform(dane['X_train'][wybrane_cechy])
10 X_test_scaled = StandardScaler().fit_transform(X_test)
11
12 model = LogisticRegression(penalty='l2', C=1.0, class_weight='balanced', solver='lbfgs',
   max_iter=2000)
13 model.fit(X_train_scaled, dane['y_train'])
14 probs = model.predict_proba(X_test_scaled)[:, 1]
15
16 n_iterations = 1000
17 boot_auc, boot_recall = [], []
18
19 # P tla generuj ca 1000 wirtualnych pr b pacjent w z oryginalnego zbioru testowego
20 for _ in range(n_iterations):
21     y_test_resampled, probs_resampled = resample(y_test, probs, replace=True)
22
23     auc = roc_auc_score(y_test_resampled, probs_resampled)
24     pred = (probs_resampled >= 0.089).astype(int)
25     tn, fp, fn, tp = confusion_matrix(y_test_resampled, pred).ravel()
26     recall = tp / (tp + fn) if (tp + fn) > 0 else 0
27
28     boot_auc.append(auc)
29     boot_recall.append(recall)
30
31 # Ekstrakcja 95% Przedzia w Ufno ci (Odrzucenie 2.5% skrajno ci z obu stron)
32 ci_low_auc, ci_high_auc = np.percentile(boot_auc, 2.5), np.percentile(boot_auc, 97.5)
33 ci_low_rec, ci_high_rec = np.percentile(boot_recall, 2.5), np.percentile(boot_recall,
   97.5)
34
35 print(f"95% CI AUC: [{ci_low_auc:.3f} - {ci_high_auc:.3f}]")
36 print(f"95% CI Recall (Czu o ): [{ci_low_rec:.3f} - {ci_high_rec:.3f}]")

```

Listing 11: Symulacja Bootstrap (1000 iteracji) dla wiodącego algorytmu.